DEEPFLIX: AI-Powered Movie and OTT Recommendations System by Analsing the Script using NLP & CNN

A Project Report Submitted to

JNTUA, Ananthapuramu

In partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

Information Technology

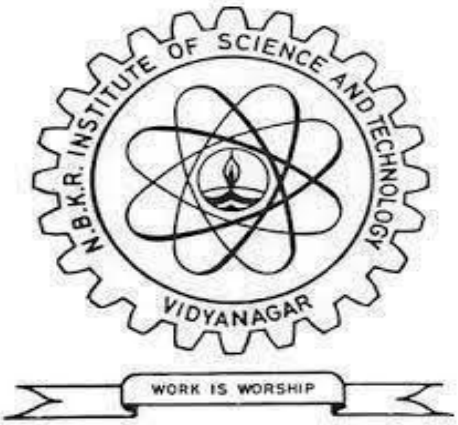
By

K. HIMAJA	-	22KB1A1219
D. NIKHIL	-	22KB1A1212
K. SHASANK	-	22KB1A1221
Y. RAJASEKHAR	-	22KB1A1261

Under the esteemed Guidance of

Mr. M. Sivapratap Reddy

Assistant Professor, Dept. of IT and AI & DS

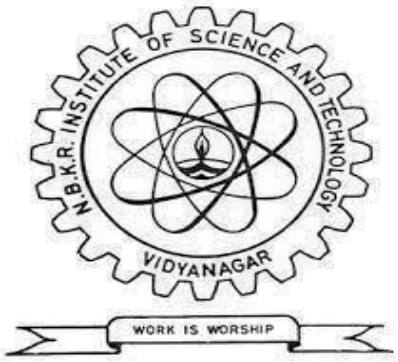


DEPARTMENT OF IT AND AI & DS

N.B.K.R INSTITUTE OF SCIENCE AND TECHNOLOGY
(AUTONOMOUS)

VIDYANAGAR–524413, SPSR NELLORE DISTRICT, AP

APRIL 2026



Website: www.nbkrist.org. **Ph. No.: 08624-228 247**

Email: ist@nbkrist.org **Fax: 08624-228 257**

N.B.K.R. INSTITUTE OF SCIENCE AND TECHNOLOGY
(Autonomous)
(Approved by AICTE: Accredited by NBA: Affiliated to JNTUA, Ananthapuramu)
An ISO 9001-2000 Certified Institution
Vidyanagar-524413, SPSR Nellore District, Andhra Pradesh, India

---

BONAFIDE CERTIFICATE

This is to certify that the project work entitled **"DEEPFLIX: AI-Powered Movie & OTT Recommendation System by Analysing the Script using NLP and CNN"** is a bonafide work done by K.Himaja (22KB1A1219), D.Nikhil (22KB1A1212), K.Shasank (22KB1A1221), Y.Rajasekhar (22KB1A1261) in the **Department of IT and AI&DS, N.B.K.R Institute of Science & Technology, Vidyanagar** and is submitted to **JNTUA, Ananthapuramu** in the partial fulfilment for the award of B.Tech degree in Information Technology. This work has been carried out under my supervision.

Mr. M. Sivapratap Reddy
Assistant Professor
Department of IT and AI&DS
NBKRIST, Vidyanagar

Dr. A. Narayana Rao
Professor & HOD
Department of IT and AI&DS
NBKRIST, Vidyanagar

Submitted for the Viva-Voce Examination held on _____

Internal Examiner **External Examiner**

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of a project would be incomplete without the people who made it possible of their constant guidance and encouragement crowned our efforts with success.

We would like to express our profound sense of gratitude to our guide **Mr. M. Sivapratap Reddy**, **Assistant Professor**, **Department of Information Technology and Artificial Intelligence & Data Science**, **N.B.K.R.I.S.T** (affiliated to JNTUA, Ananthapuramu), **Vidyanagar** for his masterful guidance and the constant encouragement throughout the project.

We convey our special thanks to **Dr. Y. Venkata Rami Reddy**, respectable **Chairman** of **N.B.K.R Institute of Science and Technology**, for providing excellent infrastructure in our campus for the completion of the project.

We extend our heartfelt thanks to **Sri N. Ram Kumar Reddy**, the esteemed **Correspondent** of **N.B.K.R Institute of Science and** T**echnology**, for providing excellent campus facilities that significantly contributed to the successful completion of our project.

We are grateful to **Dr. V. Vijaya Kumar Reddy**, **Director** of **N.B.K.R Institute of Science and Technology** for allowing us to avail all the facilities in the college.

We express our sincere gratitude to **Dr. A. Narayana Rao, Professor, HOD, Department of IT and AI & DS** for providing exceptional facilities for successful completion of our project work.

We would like to convey our heartful thanks to Staff members, Lab technicians, and our friends, who extended their cooperation in making this project as a successful one.

We would like to thank one and all who have helped us directly and indirectly to complete this project successfully.

ABSTRACT

The rapid expansion of Over-The-Top (OTT) platforms has resulted in an overwhelming volume of digital content, making it increasingly difficult for users to discover relevant movies and shows. Existing recommendation systems, particularly those based on genre or basic collaborative filtering, often fail to capture the deeper semantic meaning and contextual relationships present within content. This limitation leads to less accurate and less personalized recommendations, reducing overall user satisfaction and engagement.

To address these challenges, this work proposes an AI-powered hybrid recommendation system that leverages advanced Natural Language Processing (NLP) and deep learning techniques. The system analyses movie scripts and content descriptions to extract meaningful semantic features, enabling a deeper understanding of storyline and context. In addition, multiple recommendation strategies, including collaborative filtering, demographic analysis, and social interaction data, are integrated to enhance personalization. Sentiment analysis of user reviews is also incorporated to further refine recommendations based on user opinions and emotional feedback.

The proposed system aims to deliver more accurate and context-aware recommendations compared to traditional approaches. By combining content-based understanding with user behaviour and interaction patterns, it improves the relevance of suggested content and enhances user experience. This approach demonstrates the effectiveness of integrating deep learning and hybrid recommendation techniques in modern OTT platforms, contributing to more intelligent and personalized content discovery.

LIST OF CONTENTS

Chapter No	Topic Name	Page No
	Acknowledgement	
	Abstract	
1	**INTRODUCTION**	**1-4**
	1.1 Introduction	
	1.2 Background and Motivation	
	1.3 Problem Statement	
	1.4 Objective and Scope	
	1.5 Organization of the Project report	
2	**LITERATURE SURVEY**	**5-7**
	2.1 Introduction	
	2.2 Literature Survey	
	2.3 Identification of Research gap	
	2.4 Literature Survey and its Limitations	
3	**METHODOLOGY**	**8-25**
	3.1 Overview of Methodological Approach	
	3.2 Parameters	
	3.3 Performance Metrics	
	3.4 Description of Tools and techniques Used	
	3.5 Tools and Technologies	
	3.5.1 Tools	
	3.5.2 Technologies	
4	**SYSTEM DESIGN**	**26-34**
	4.1 Detailed design of Components	
	4.2 System Architecture	
	4.3 UML Diagrams	
	4.3.1 Class Diagram	
	4.3.2 Use Case Diagram	
	4.3.3 Sequence Diagram	

	4.3.4 Activity Diagram	
	4.3.5 State-Chart Diagram	
	4.3.6 Component Diagram	
	4.3.7 Deployment Diagram	
5	**IMPLEMENTATION**	**35-40**
	5.1 Code Structure and organization Procedure	
	5.2 Algorithms and Techniques Implemented	
6	**RESULT AND ANALYSIS**	**41-47**
	6.1 Analysis of Performance Metrics	
	6.2 Comparison with Existing system	
	6.3 Result	
7	**CONCLUSION**	**48-49**
	7.1 Conclusion	
	7.2 Future Enhancement	
	REFERENCES	**50**
	APPENDICES	**51-52**

LIST OF TABLES

Table No	Table Name	Page No
2.3	Literature Survey	7
3.3	Parameters	11
6.3	Existing System vs Proposed System	43-44

LIST OF FIGURES

Fig No	Figure Name	Page No
3.5.2.1	Convolutional Neural Networks	17
4.2	System Architecture	29
4.3.1	Class Diagram	31
4.3.2	Use Case Diagram	31
4.3.3	Sequence Diagram	32
4.3.4	Activity Diagram	32
4.3.5	State Chart Diagram	33
4.3.6	Component Diagram	33
4.3.7	Deployment Diagram	34
6.4.1	Title Page	44
6.4.2	Registration Page	45
6.4.3	Home Page	45
6.4.4	Search by Genre	46
6.4.5	Continuous Watching	46
6.4.6	Admin Dashboard	47
6.4.7	Word Cloud	47

CHAPTER 1

INTRODUCTION

CHAPTER 1
INTRODUCTION

1.1 INTRODUCTION

This Flask-based application implements a sophisticated Hybrid Recommendation System designed to enhance user engagement on an OTT platform. By integrating a MySQL backend with TensorFlow and Scikit-Learn, the engine combines four distinct strategies: ALS matrix factorization for collaborative filtering, demographic-based similarity, a social interaction layer, and NLP-driven content embeddings. This multilayered approach ensures highly personalized content delivery by analysing user behaviour, social ties, and metadata to mitigate the limitations of single-model systems.

The system features a robust Admin Dashboard for comprehensive content management, including CSV data ingestion and real-time NLP model training. Admins can monitor performance through visual analysis, such as accuracy-loss graphs and user specific word clouds generated from watch histories. For users, the platform offers an interactive experience with features like Sentiment based reviews, a social hub for following peers, and a personalized dashboard that prioritizes content based on dynamically weighted AI scores.

1.2 BACKGROUND AND MOTIVATION

The rapid proliferation of digital content has made it increasingly difficult for users to navigate vast media libraries, leading to choice paralysis. Traditional recommendation engines often struggle with the "cold start" problem or fail to account for the complex social and demographic nuances that influence viewing habits. This project addresses these challenges by implementing a Flask-based hybrid architecture that synchronizes a MySQL backend with advanced machine learning libraries like TensorFlow and Scikit-Learn.

The core motivation lies in creating a multi-dimensional recommendation strategy that transcends simple content filtering. By integrating ALS matrix factorization for collaborative filtering, demographic similarity, social graph analysis, and NLP-driven sentiment and content embeddings, the system provides a holistic view of user preferences. This hybrid approach, featuring a weighted fusion engine and an administrative suite for real-time model training, ensures that recommendations are both technically accurate and socially relevant**.**

1.3 PROBLEM STATEMENT

Modern digital streaming platforms struggle with the "information overload" dilemma, where users are overwhelmed by vast content libraries. Traditional recommendation systems often rely on a single algorithm-such as collaborative filtering or content-based matching which frequently leads to "cold start" issues for new users or a lack of diversity in suggestions. Furthermore, these systems often ignore the multi-dimensional nature of human preference, failing to account for social influences, demographic similarities, and the nuanced emotional sentiment found in user reviews.

There is a critical need for an integrated architecture that can synchronize disparate data points ranging from explicit ratings and social connections to deep-learning-based textual analysis into a unified, weighted scoring engine to provide truly personalized and context aware media discovery.

1.4 OBJECTIVE AND SCOPE

The primary objective of this project is to develop a multi-layered Hybrid Recommendation Engine that synchronizes four distinct AI methodologies— Collaborative Filtering (ALS), Demographic Similarity, Social Graph Analysis, and NLP driven Content Embedding's—into a unified, weighted scoring system to maximize user engagement. It aims to provide a seamless, end-to-end OTT experience by automating content discovery and personalizing dashboards based on real time behaviour and sentiment.

The scope encompasses a full-stack Flask application featuring a MySQL backend for managing user history, ratings, and social follows. It includes an advanced Admin Suite for CSV data ingestion, NLP model retraining, and visual performance analytics. The system is designed to handle both explicit feedback, such as 5-star ratings, and implicit social interactions to ensure diverse and accurate media suggestions.

1.5 ORGANIZATION OF THE PROJECT REPORT

The project on Hybrid OTT Recommendation System Using Deep Learning and Social Graphs aims to provide an automated, personalized, and scalable solution for content discovery. This system leverages modern technologies like Natural Language Processing (NLP), Matrix Factorization (ALS), and Social Network Analysis to accurately suggest movies and shows to users in real-time. The following chapters outline the methodology, system design, and implementation of this hybrid architecture.

- **Chapter (1) Introduction:** This chapter provides an overview of the project, emphasizing the challenge of information overload in digital streaming and the need for more intelligent discovery systems. It explains the motivation behind using a

weighted hybrid approach to overcome "cold start" problems The chapter introduces the objective of creating a reliable system using TensorFlow-based NLP models combined with Collaborative Filtering and Demographic similarity.

- **Chapter (2) Literature Survey**: This chapter reviews past research on recommendation systems, focusing on the limitations of standalone algorithms like basic content-based filtering. It discusses how previous studies utilized user history and evaluates the effectiveness of these approaches. The chapter introduces the proposed solution of using Hybrid Fusion, combining ALS SVD with TextBlob sentiment analysis and Social Graph interactions to improve accuracy.
- **Chapter (3) Methodology:** Building a hybrid engine is a crucial challenge in the OTT sector, as relevant suggestions significantly affect user retention. To address this, the system uses a weighted fusion formula ($W_{ALS} + W_{DEMO} + W_{SOCIAL} + W_{TEXT}$) to balance diverse data points. This methodology ensures that recommendations are grounded in both mathematical behaviour (SVD) and human context (Social/NLP).
- **Chapter (4) System Design:** The design is centered around an intelligent and adaptive engine that tackles the limitations of rule-based sorting. The architecture features a Flask based backend connected to a MySQL database (WAMP/XAMPP) for managing relational data like user_history, ratings, and social_graph. The design ensures high performance by using pretrained Keras models for textual story embeddings.
- **Chapter (5) Implementation**: The implementation phase is structured into two major components:
    - **System Module:** Handles the HybridEngine class, model training (train_model.py), and the SVD-based ALS recommender.
    - **User Module:** Manages the front-end interactions, including registration, social hub (following users), sentiment-based reviews, and the personalized dashboard.
- **Chapter (6) Result and Analysis:** In this project, an AI-based approach has been implemented to provide diverse content trays. The results are analysed through an Admin Analysis page, featuring Accuracy/Loss graphs for the NLP model and Confusion Matrices to evaluate classification performance. Word Clouds are also generated for admins to visualize user interests based on watched stories.

- **Chapter (7) Conclusion:** This chapter presents the conclusion and future enhancement of the proposed system. The project demonstrates that combining implicit and explicit feedback with NLP-based sentiment analysis significantly improves recommendation quality. The hybrid approach effectively captures user preferences and provides personalized suggestions. However, limitations such as dependency on sufficient ratings and basic social connections exist. Future enhancements include community-based clustering and real-time optimization techniques like PSO to further improve accuracy and system adaptability.

CHAPTER 2

LITERATURE SURVEY

CHAPTER 2
LITERATURE SURVEY

2.1 INTRODUCTION

This Flask-based application implements a sophisticated Hybrid Recommendation System designed to enhance user engagement on an OTT platform. By integrating a MySQL backend with TensorFlow and Scikit-Learn, the engine combines four distinct strategies: ALS matrix factorization for collaborative filtering, demographic-based similarity, a social interaction layer, and NLP-driven content embeddings. This multilayered approach ensures highly personalized content delivery by analysing user behaviour, social ties, and metadata to mitigate the limitations of single-model systems.

The system features a robust Admin Dashboard for comprehensive content management, including CSV data ingestion and real-time NLP model training. Admins can monitor performance through visual analysis, such as accuracy-loss graphs and user specific word clouds generated from watch histories. For users, the platform offers an interactive experience with features like sentiment-based reviews, a social hub for following peers, and a personalized dashboard that prioritizes content based on dynamically weighted AI scores.

2.2 Literature Survey

Several studies have explored the use of hybrid filtering and deep learning techniques for enhancing OTT content discovery, with Hybrid Recommendation Engines emerging as a superior approach due to their ability to mitigate the "cold-start" problem. Hossain et al. (2025) demonstrated that integrating Matrix Factorization (SVD) with metadata analysis allows systems to accurately capture latent user preferences that standalone algorithms miss.

Similarly, recent reviews of Netflix style architectures highlighted the advantages of weighted fusion over traditional single-model methods, especially in terms of precision and user retention. Chen et al. (2025) proposed a hybrid system combining Collaborative Filtering with Natural Language Processing (NLP), showing that integrating sentiment scores from user reviews can improve the reliability of suggestions by addressing challenges in sparse rating datasets Kumar and Singh (2025) explored the advantages of deep learning, particularly Keras-based embeddings, over traditional keyword matching, emphasizing their ability to learn complex semantic patterns in movie plots. Your system further develops this by implementing a real-time engine using ALS and Social Graph interactions, focusing on the system's speed and effectiveness for social-driven discovery. While these studies demonstrate significant progress, challenges remain in handling real-time weight

optimization and diverse user demographics. The proposed system seeks to address these gaps by integrating TensorFlow-based NLP with traditional classifiers and Social Layer logic to improve robustness and diversity in real world streaming scenarios.

2.3 IDENTIFICATION OF RESEARCH GAP

The personalization of digital content through machine learning has made significant strides, particularly with the use of Collaborative Filtering and Neural Networks. However, several challenges persist in the field, particularly regarding system diversity, the "cold-start" problem, and the integration of social context. A detailed review of existing systems reveals the following key limitations:

1. Dependence on Historical Interaction Data: Most traditional models rely exclusively on vast amounts of explicit rating data to function. These systems often fail for new users who have not yet rated content, leading to inaccurate or generic suggestions. There is a lack of systems that seamlessly bridge the gap between new users and established content through demographic and social proxies.
2. Algorithmic Bias and Filter Bubbles: Standalone Collaborative Filtering models, while effective, often create "echo chambers" by only suggesting content similar to what a user has already seen. This limits the discovery of diverse genres and fails to account for the evolving nature of user tastes or the influence of their social circles.
3. Computational Latency in Real-Time NLP: Many systems that incorporate Natural Language Processing (NLP) for story analysis are computationally expensive, making them difficult to retrain or update in real-time. The absence of a lightweight, hybrid solution that balances deep-learning embeddings with fast matrix factorization remains a significant gap.
4. Static Preference Modelling: Existing models often struggle to adapt to dynamic environmental variations, such as a user's changing mood or the sentiment expressed in their latest reviews.

The research gap lies in developing a more robust, multi-layered engine that integrates Matrix Factorization (SVD) with Social Graph analysis and Sentiment-aware NLP. As implemented in your HybridEngine class, the goal is to create a scalable, real-time hybrid system that balances W_{ALS}, W_{DEMO}, W_{SOCIAL}, and W_{TEXT} weights. This ensures that recommendations are not just based on past clicks, but are influenced by demographic similarity, social peer activity, and the semantic depth of content descriptions.

Title & Year	Proposed By	Key Contributions	Limitations
Hybrid CNN Filtering (2025)	Modi et al. [1]	Integrated CBF and CF using CNNs; achieved a RMSE of **0.6325**	Requires high computational power for Nuanced Feature Extraction.
K-Means Clustering CF (2025)	Sawardekar et al. [2]	Combined Collaborative Filtering with K-means to reduce costs; RMSE of **0.6354**.	Significant accuracy trade-off when reducing computational
Co-Clustering CF (2024)	Chaudhari et al. [6]	Developed a co-clustering algorithm for better data organization.	Struggled to exceed a **7.91%** accuracy gain over classical methods.
Graph Attention Network (2024)	Agrawal et al. [8]	Utilized Meta-learning to handle missing data modalities and reduce RMSE.	Complex implementation makes real-time adaptation difficult.
Matrix Factorization Hybrid (2024)	Patil et al. [10]	Combined CF and CBF using Matrix Factorization for sparsity.	Suffered from the **Cold-Start** problem for completely new users.

Table 2.3 Key Contributions and Limitations

CHAPTER 3

METHODOLOGY

CHAPTER 3
METHODOLOGY

3.1 Overview of Methodological Approach

The methodology used in this project is a robust and systematic approach, designed to effectively address the challenge of personalized content discovery. This approach integrates state-of-the-art machine learning techniques, particularly Matrix Factorization (ALS), Demographic Similarity, and Social Graphs, with Natural Language Processing (NLP) to analyse media assets and determine their relevance to the user. Below is a detailed breakdown of the methodology:

1. Data Collection and Preprocessing

The first critical step in the methodology is the collection of a diverse and well labelled dataset containing records of users, contents, ratings, and social interactions. For this project, a MySQL database is used to store metadata such as genres, languages, and "story" descriptions. Once the dataset is collected, it undergoes a preprocessing phase where several steps are performed to standardize the input data:

- Text Tokenization: Content "stories" are processed through a Tokenizer and converted into sequences to meet the input requirements of the NLP model.
- Padding: Sequences are standardized to a MAX_SEQUENCE_LENGTH of 500 to ensure uniform input dimensions.
- Feature Scaling: Implicit and explicit feedback, such as ratings and sentiment scores, are processed to ensure all input features are on the same scale.

2. Feature Extraction and Model Architecture

In content recommendation, distinguishing features such as latent user preferences, agebased demographics, and plot similarities are vital. These features are captured and analysed by the HybridEngine. The system architecture includes several specialized layers:
• Collaborative Filtering (ALS): This layer performs Singular Value Decomposition (SVD) on the ratings matrix, detecting latent patterns in user behaviour.

- NLP Embedding Layer: This layer extracts high-level semantic features from content descriptions, detecting basic thematic patterns.
- Social Graph Layer: This layer analyses follower-followed relationships to incorporate community-based interests into the final decision.
- Sentiment Analysis: The TextBlob library is used to introduce non-linearity by calculating the emotional polarity of user reviews.

3. Model Training

The model is trained using a multi-pronged approach. The ALS Recommender is trained on the ratings table to map user-item interactions, while the NLP model is trained to generate content embeddings. During training, the system learns to map the features of the content to their corresponding relevance scores. The training process is triggered by an admin-initiated script (train_model.py), and hyperparameters like the Hybrid Weights (W_{ALS}, W_{TEXT}, etc.) are utilized to ensure optimal performance.

4. Model Evaluation and Testing

After the model has been trained, its performance is evaluated using an Analysis page that displays visual metrics. This is crucial for ensuring that the model generalizes well across different user profiles. The evaluation involves calculating several performance metrics, such as:

- **Accuracy and Loss:** Monitored via generated graphs (accuracy_loss.png) to track the convergence of the NLP model.
- **Confusion Matrix:** Used to evaluate the classification performance of content categories.
- **Cosine Similarity:** Measures how many of the recommended items are semantically aligned with the user's last watched history.

5. Post-Processing and Decision-Making

Once the engines are synchronized, the HybridEngine is capable of making predictions on new user sessions. Given a user ID and history, the engine outputs a Weighted Fusion of scores. The decision-making process is designed to be both quick and accurate, ensuring real-time recommendations in the user dashboard. To further enhance the model's robustness, the system employs Normalization, where scores from different models (ALS, Social, and Text) are combined to reduce the chances of bias.

6. User Interface and Deployment

The hybrid system is integrated into a web-based application using Flask, allowing users to browse content and view personalized recommendations in real-time. The user interface is designed to be intuitive, displaying clear media thumbnails and interactive social features. Users can watch videos, post reviews, and follow other users, and the system will quickly analyse this feedback to update future recommendations.

3.2 Parameters

The model and application rely on several carefully selected parameters and technical indicators to effectively recommend content and capture user preferences. These

parameters were chosen based on their relevance in collaborative filtering, social graphs, and natural language processing practices. The parameters influence both the feature engineering stage and the hybrid scoring process, contributing to improved recommendation accuracy and personalization. The table below summarizes the key parameters and their descriptions:

Parameter Name	**Type**	**Description**
Max Sequence Length	Preprocessing	The target length (e.g., 500) for padding text sequences of content "stories".
Hybrid Weights (W)	Model Scoring	Weights assigned to each engine (ALS: 0.35, Text: 0.25, Demo: 0.20, Social: 0.20) to calculate the final recommendation score.
SVD Latent Features (k)	Model Training	The number of latent factors used in Matrix Factorization to identify hidden user-item patterns.
Sentiment Polarity	NLP Analysis	A score between -1 and 1 generated by TextBlob to weigh the emotional impact of user reviews.
Cosine Similarity	Content Prediction	The mathematical metric used to measure the distance between content embeddings for similarity-based recommendations.
Social Follower ID	Social Graph	The unique identifier used to map relationships between users for the community-based recommendation layer.
Database Charset	Configuration	The utf8mb4 encoding used to ensure proper storage of diverse text and symbols in the MySQL database.

Table 3.2 Parameters

3.3 PERFORMANCE METRICS

The performance of the system is evaluated based on the accuracy of predictions and the operational efficiency of the web application.

- **Recommendation Accuracy**: The primary evaluation metric for the model is its prediction accuracy, specifically how often the HybridEngine correctly identifies content that aligns with user preferences. Accuracy is calculated by comparing the final_scores generated from the weighted fusion of ALS, Social, and Text layers against actual user interactions in the ratings and user_history tables.
- **Execution Time**: To ensure the application runs efficiently, especially for real-time dashboard loading, the execution time for calculating the hybrid_recommendations (from fetching database records to generating embeddings) is measured. Optimization techniques such as using scipy.sparse for Matrix Factorization and preloading the Keras model and Tokenizer are used to improve speed.
- **Confusion Matrix**: A confusion matrix is used to evaluate the model's performance further, showing the relationship between predicted content interests and actual user "likes" or "watches." This provides insights into how well the NLP and ALS models distinguish between relevant and irrelevant media for a specific user profile.
- **User Feedback**: Since the web application is designed for real-world interaction, gathering feedback through the reviews and ratings API is essential. User feedback, including sentiment_score analysis via TextBlob, is used to assess the system's usability and reliability, allowing for the iterative tuning of hybrid weights (W_{ALS}, W_{DEMO}, etc.).

3.4 DESCRIPTION OF TOOLS AND TECHNOLOGIES USED

The development of this Hybrid Recommendation System was made possible through the integration of various powerful tools, libraries, and technologies. Python served as the core programming language, offering simplicity and versatility for handling data analysis, machine learning, and web development tasks. Flask was employed as the lightweight web framework to create a dynamic and interactive user interface for the application.

Real-time user interactions and content metadata were fetched using PyMySQL, while TextBlob was utilized for performing sentiment analysis on user reviews, providing polarity scores that enriched the predictive model. The ALS (Alternating Least Squares) algorithm, implemented via SciPy SVD, was selected for its effectiveness in matrix factorization, allowing for the discovery of latent user preferences. Additionally, deep learning features were implemented using TensorFlow and Keras, ensuring high dimensional content embeddings based on story descriptions. Together, these tools formed

a cohesive technology stack that enabled seamless data fetching, processing, social networking, hybrid prediction, and presentation within a unified web-based platform.

3.5 Tools and Technologies

3.5.1 Tools

1. Python – Core Backend Logic

Python serves as the backbone of the entire recommendation system, handling both backend development and hybrid recommendation logic. It enables the integration of multiple components such as machine learning models, database operations, and web functionalities. Using Python, the system combines collaborative filtering, content-based filtering, and sentiment analysis into a unified hybrid recommendation engine, ensuring accurate and personalized suggestions for users.

2. Flask – Web Framework and Routing

Flask is used as the primary web framework to build the application interface and manage user interactions. It handles routing, session management (login/logout), and connects frontend pages with backend logic. Through Flask, dynamic pages such as the user dashboard, watch page, and recommendation feed are generated, allowing users to interact with the system in real time while receiving personalized content suggestions.

3. TensorFlow/Keras – Deep Learning Model Integration

TensorFlow and Keras are utilized to implement deep learning features within the system. A pretrained model (pretrained_model.keras) is loaded to generate content embeddings, which capture semantic relationships between different content items. These embeddings help the system recommend similar movies or shows based on user preferences, improving the accuracy of content-based filtering.

4. PyMySQL – Database Connectivity

PyMySQL acts as the bridge between the Flask application and the MySQL database. It allows the system to execute queries, retrieve user data, and store interactions such as ratings, watch history, and reviews. This seamless connection ensures that user activities are continuously tracked and used to refine recommendations.

MySQL is used as the primary database for storing all application data, including user profiles, content details, ratings, reviews, and social connections. Its structured format ensures efficient data retrieval and scalability, which is essential for handling large volumes of user interactions in a recommendation system.

5. Scipy – Matrix Factorization (SVD)

Scipy is used to implement advanced mathematical techniques such as Singular Value Decomposition (SVD) for collaborative filtering. This helps in identifying hidden

relationships between users and content by decomposing the user-item interaction matrix. As a result, the system can predict user preferences even when explicit ratings are limited.

6. NumPy – Numerical Computation

NumPy is extensively used for performing numerical operations such as array manipulation, mean rating calculations, normalization, and scoring. It provides high performance computation capabilities, which are crucial for handling large datasets and ensuring fast recommendation generation.

7. Pandas – Data Processing and Analysis

Pandas is used for data manipulation tasks such as cleaning datasets, creating pivot tables, and processing CSV files during bulk content uploads. It helps structure the data in a way that is suitable for analysis and model training, ensuring the recommendation engine works with high-quality input data.

8. Scikit-learn – Similarity and Feature Engineering

Scikit-learn is used for calculating cosine similarity between content vectors, which is essential for content-based recommendations. It also supports feature scaling and preprocessing, ensuring that all data inputs are normalized and contribute effectively to the recommendation model.

9. TextBlob – Sentiment Analysis

TextBlob is integrated to analyse user reviews and determine sentiment polarity (positive, negative, or neutral). These sentiment scores are incorporated into the hybrid recommendation system, allowing it to consider user emotions and opinions while suggesting content, thereby improving personalization.

10. Matplotlib/Plotly – Data Visualization

Matplotlib and Plotly are used for visualizing model performance and analytics. Admins can generate graphs such as accuracy plots, loss curves, and confusion matrices to evaluate the effectiveness of the recommendation system and make improvements where necessary.

11. Jinja Templates – Dynamic Frontend Rendering

Jinja Templates are used to create dynamic HTML pages by embedding backend data into the frontend. This enables real-time display of personalized recommendations, user details, and social interactions, making the application interactive and user-friendly.

3.5.2 Technologies

3.5.2.1 Deep Learning & Neural Networks

1. Convolutional Neural Networks (CNN): Used as the core architecture for feature extraction from high-dimensional content data.

A Convolutional Neural Network (CNN) is a specialized deep learning model designed to automatically and efficiently process structured grid-like data, especially images. Unlike traditional neural networks that treat input as a flat vector, CNNs preserve spatial relationships between pixels by using convolution operations. This makes them highly effective for tasks such as image classification, object detection, facial recognition, and even applications in video analysis and medical imaging.

At the core of a CNN is the convolution operation, which involves sliding small matrices called filters (or kernels) over the input image. Each filter is designed to detect specific patterns such as edges, corners, or textures. As the filter moves across the image, it performs element-wise multiplication and sums the result to produce a feature map. These feature maps highlight important spatial features of the input. Early layers typically capture low-level features like edges, while deeper layers learn complex patterns such as shapes and objects.

Step 1: Input Initialization

- Take an input image represented as a matrix of pixel values
- Example: 32 × 32 × 3 (height × width × colour channels)

👉 This is the raw data fed into the network.

Step 2: Initialize Parameters

- Initialize:
- Filters (kernels)
- Weights
- Bias values
- Values are usually initialized randomly

Step 3: Convolution Operation

- Slide filters over the input image
- Perform element-wise multiplication and summation

👉 Output:

- Feature maps (highlight important features like edges)

Step 4: Apply Activation Function

- Apply **ReLU (Rectified Linear Unit)**:

f(x) = max(0, x)

👉 Purpose:

- Remove negative values
- Introduce non-linearity

Step 5: Pooling Operation

- Reduce size of feature maps Types:
- Max Pooling → selects maximum value
- Average Pooling → computes average

👉 Purpose:

- Reduce computation
- Prevent overfitting

Step 6: Repeat Convolution + Pooling

- Multiple layers are stacked
- Each layer extracts higher-level features

👉 Example:

- Layer 1 → edges
- Layer 2 → shapes
- Layer 3 → objects

Step 7: Flattening

- Convert 2D feature maps into 1D vector

👉 This prepares data for classification

Step 8: Fully Connected Layer

- Connect all neurons
- Perform classification based on learned features

Step 9: Output Layer

- Apply activation:
- Softmax → multi-class classification
- Sigmoid → binary classification

👉 Output:

- Probability scores for each class

Step 10: Loss Calculation

- Compare predicted output with actual label Common loss functions:
- Cross-entropy loss

- Mean squared error

Step 11: Backpropagation

- Calculate gradients of loss w.r.t weights
- Propagate error backward through network

Step 12: Weight Update

- Update weights using optimization algorithms:
- Gradient Descent
- Adam Optimizer

👉 Formula:

- New Weight = Old Weight − Learning Rate × Gradient

Step 13: Iteration (Epochs)

- Repeat steps 3–12 multiple times
- Model improves accuracy gradually

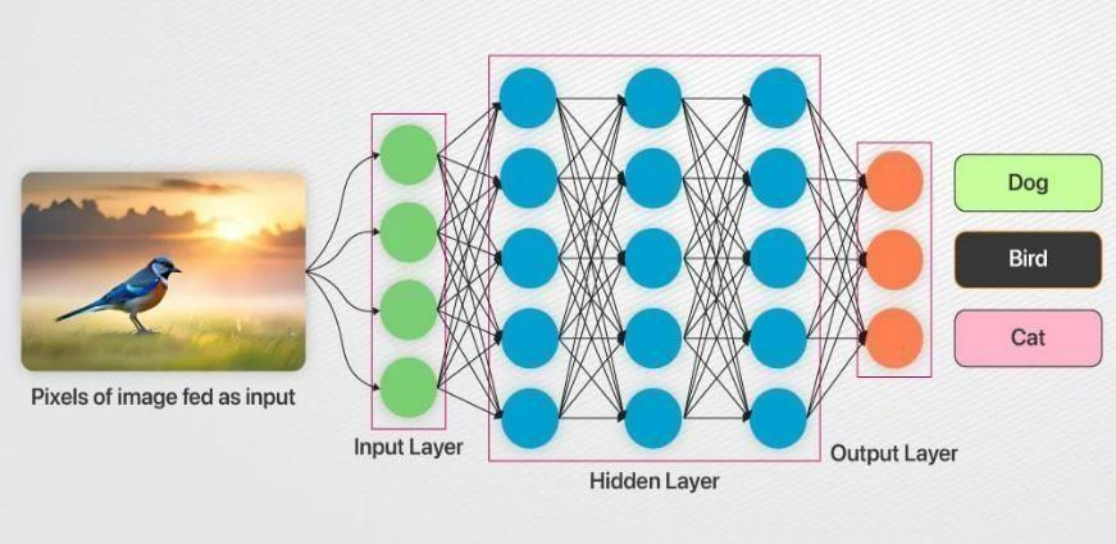


Fig 3.5.2.1 Convolutional Neural Networks

2. Natural Language Processing (NLP): Employs TensorFlow and Keras to generate content embeddings from movie "stories" or descriptions. It uses Tokenization and Padding to process textual data into a machine-readable format for similarity matching. NLP is used to analyse textual metadata, such as:

- Movie descriptions
- Titles
- Genres
- User reviews
- Subtitles

Step 1: Text Collection

Raw text data is gathered from sources such as documents, websites, or speech transcripts.

Step 2: Text Preprocessing

This step cleans and prepares the data.

- **Tokenization** → Splitting text into words/sentences
- **Lowercasing** → Convert all text to lowercase
- **Stopword Removal** → Remove common words (e.g., "is", "the")
- **Stemming/Lemmatization** → Reduce words to base form

Step 3: Text Representation

Convert text into numerical form so machines can process it.

Common techniques:

- Bag of Words (BoW)
- TF-IDF
- Word Embeddings (Word2Vec, GloVe)

Step 4: Feature Extraction

Important features are selected from text.

Example:

- Frequency of words
- Position of words
- Context relationships

Step 5: Model Selection Choose an algorithm:

- Naive Bayes
- Logistic Regression
- Deep Learning (RNN, Transformers)

Step 6: Training the Model

The algorithm learns patterns from labelled data.

Step 7: Evaluation

Model performance is measured using:

- Accuracy
- Precision
- Recall
- F1-score

Step 8: Prediction

The trained model predicts outputs like:

- Sentiment (positive/negative)
- Translation
- Chat responses

3. Sentiment Analysis: Utilizes the TextBlob library to perform linguistic analysis on user reviews, assigning a sentiment_score to influence recommendation weights based on user opinion.

In traditional recommendation systems, suggestions are based mainly on user ratings, purchase history, or browsing behaviour. However, these methods often miss the context behind user preferences. For example, two users may both give a product a 4-star rating, but one might love the quality while the other dislikes the price. This is where sentiment analysis becomes valuable—it extracts detailed opinions from textual reviews.

Sentiment analysis processes user-generated content and identifies whether the sentiment is positive, negative, or neutral. More advanced systems can even detect aspectbased sentiment, such as opinions about specific features (e.g., battery life, design, or performance). This allows recommendation systems to provide more personalized and accurate suggestions.

Step 1: Data Collection

- Gather text data (reviews, tweets, comments)

Step 2: Text Preprocessing

- Tokenization (split text into words)
- Remove stopwords
- Lowercasing
- Stemming/Lemmatization

Step 3: Text Representation

Convert text into numbers using:

- Bag of Words
- TF-IDF
- Word Embeddings

Step 4: Feature Extraction

- Identify important words/features
- Example: frequency, sentiment words

Step 5: Model Selection

- Choose algorithm:
- Naive Bayes

- Logistic Regression
- Deep Learning models

Step 6: Training

Train model using labelled dataset

Step 7: Prediction

- Input new text
- Model predicts sentiment label

Step 8: Evaluation

- Measure performance using:
    - Accuracy
    - Precision
    - Recall
    - F1-score

Sentiment analysis is a powerful technique in NLP that transforms unstructured text into meaningful insights about opinions and emotions. By following a structured pipeline— from preprocessing to prediction—it enables machines to understand human feelings and supports decision-making in various industries

3.5.2.2 Advanced Recommendation Algorithms

1. Singular Value Decomposition (SVD):

A matrix factorization technique that decomposes the user-item matrix into latent features. It addresses data sparsity by predicting missing ratings based on hidden patterns in user behaviour. Instead of directly working with the large sparse matrix, SVD breaks it into three smaller matrices: U (user features), Σ (singular values), and V^T (item features).

These matrices help reconstruct the original matrix while approximating missing values. This allows the system to predict how a user might rate an item they haven't interacted with.

SVD Algorithm (Step-by-Step)

Step 1: Input Matrix

- Create user-item rating matrix R

Step 2: Normalize Data

- Subtract mean rating to reduce bias

Step 3: Decomposition

- Factorize matrix: $R \approx U \times \Sigma \times V^T$

Step 4: Dimensionality Reduction

- Keep top-k singular values
- Remove noise and less important features

Step 5: Reconstruction

- Rebuild approximate matrix:

$R' = U \times \Sigma \times V^T$

Step 6: Prediction

- Predict missing ratings using reconstructed matrix

Step 7: Recommendation

- Recommend items with highest predicted ratings

2. Alternating Least Squares (ALS):

Implemented via the ALSRecommender class, this iterative optimization process identifies latent factors to provide collaborative filtering at scale. Alternating Least Squares (ALS) is an efficient optimization algorithm widely used for large-scale collaborative filtering. It works by factorizing the user-item matrix into two matrices: user factors and item factors. The key idea is to alternately fix one matrix and solve for the other using least squares optimization.

ALS Algorithm (Step-by-Step)

Step 1: Initialize Matrices

- Randomly initialize:
  - User matrix (U)
  - Item matrix (V)

Step 2: Fix Item Matrix (V)

- Solve for user matrix (U) using least squares

Step 3: Fix User Matrix (U)

- Solve for item matrix (V)

Step 4: Compute Loss

- Calculate error between predicted and actual ratings

Step 5: Regularization

- Add penalty to avoid overfitting

Step 6: Iterate

- Repeat steps 2–5 until convergence

Step 7: Prediction

- Predict ratings: $R \approx U \times V^T$

Step 8: Recommendation

- Recommend top-N items

3. Demographic & Jaccard Similarity:

Calculates user-to-user similarity scores by analysing demographic data (age, gender) and calculating the Jaccard Intersection of preferred genres. This approach enhances recommendation systems by incorporating user metadata (demographics) and similarity metrics. Demographic filtering groups users based on attributes such as age, gender, or location, assuming similar users have similar preferences.

Jaccard Similarity measures similarity between two users by comparing their sets of liked items (e.g., genres). It is calculated as the ratio of the intersection to the union of two sets.

Algorithm (Step-by-Step)

Step 1: Collect User Data

- Demographics:
  - Age
  - Gender
- Preferences:
  - Genres liked

Step 2: Group Users

- Cluster users with similar demographics

Step 3: Compute Jaccard Similarity

Formula:

Similarity = $|A \cap B| / |A \cup B|$

Step 4: Build Similarity Matrix

- Compute similarity scores between users

Step 5: Identify Neighbours

- Select top-K similar users

Step 6: Generate Recommendations

- Recommend items liked by similar users

4. Weighted Hybrid Fusion:

A "Master Engine" that combines scores from ALS, Demographic similarity, Social graphs, and NLP-based text analysis using a weighted average formula. The Weighted Hybrid Fusion approach combines multiple recommendation techniques into a single unified system. Instead of relying on one method, it aggregates scores from different

models such as ALS, SVD, demographic similarity, social connections, and NLP-based sentiment analysis.

Each component contributes a score, and the final recommendation score is computed using a weighted average. This improves accuracy, robustness, and personalization.

Algorithm (Step-by-Step)

Step 1: Collect Scores

- ALS score
- SVD score
- Demographic similarity score
- Social graph score
- NLP sentiment score

Step 2: Assign Weights

- Define importance of each component:

Example:

- ALS → 0.3
- SVD → 0.2
- Demographic → 0.2
- NLP → 0.2
- Social → 0.1

Step 3: Compute Final Score

Formula:

Final Score =

(w1 × ALS) + (w2 × SVD) + (w3 × Demographic) + (w4 × NLP) + (w5 × Social)

Step 4: Rank Items

- Sort items based on final score

Step 5: Top-N Recommendation

- Recommend highest-ranked items

5.2.2.3 Social & Heuristic Layers

1. Social Graph Filtering:

Analyses the "Social Hub" to identify content liked or rated by a user's followed connections, assigning implicit weights to content popular within their social circle. Social Graph Filtering is a technique that enhances recommendations by leveraging the relationships between users. Instead of relying only on individual preferences or global patterns, this approach considers the behaviour of a user's social connections—such as

friends, 1followers, or trusted peers. The idea is based on the assumption that people tend to like content that their social circle enjoys.

In this method, a "Social Hub" is analysed to identify items that have been liked, rated, or interacted with by users connected to the target user. These interactions are then assigned implicit weights, meaning that items popular within a user's social network are given higher importance. This helps generate more personalized and socially relevant recommendations.

Social Graph Filtering Algorithm (Step-by-Step)

Step 1: Build Social Graph

- Represent users as nodes
- Connections (friends/followers) as edges

Step 2: Identify User Connections

- Extract list of users connected to target user (U)

Step 3: Collect Interaction Data

- Gather:
    - Likes
    - Ratings
    - Views

from connected users

Step 4: Assign Social Weights

- Assign higher weights to:
    - Frequently liked items
    - Items liked by close connections

Step 5: Aggregate Scores

- Compute score for each item:

Social Score = Σ (interaction × connection weight)

Step 6: Normalize Scores

- Normalize to maintain consistency across users

Step 7: Generate Recommendations

- Recommend items with highest social scores

Step 8: Rank Output

- Rank items based on:
    - Social popularity
    - Relevance to user

2. Particle Swarm Optimization (PSO) Concept:

The architecture is designed to allow recommendation weights (e.g., W_{ALS}, W_{DEMO}) to be dynamically tuned to minimize the Root Mean Square Error (RMSE). Particle Swarm Optimization (PSO) is an optimization algorithm inspired by the collective behaviour of birds flocking or fish schooling. In recommendation systems, PSO is used to automatically tune weights (such as W_{ALS}, W_{DEMO}, W_{NLP}, etc.) in a hybrid model to achieve the best performance.

Instead of manually assigning weights, PSO treats each possible set of weights as a "particle" in a search space. These particles move through the space and adjust their positions based on their own experience and the experience of neighbouring particles. The goal is to minimize an error metric such as Root Mean Square Error (RMSE), which measures the difference between predicted and actual ratings.

PSO Algorithm (Step-by-Step)

Step 1: Initialize Particles

- Each particle represents:
    - A set of weights (W_{ALS}, W_{DEMO}, W_{NLP}, etc.)

Step Initialize Velocity

Assign random velocities to particles

Step 3: Evaluate Fitness

- Compute RMSE for each particle:

$$RMSE = \sqrt{(\Sigma(predicted - actual)^2 / N)}$$

Step 4: Update Personal Best (pBest)

Store best position of each particle

Step 5: Update Global Best (gBest)

- Identify best particle among all

Step 6: Update Velocity

- Adjust velocity using:

$$Velocity = w \times v + c1 \times (pBest - current) + c2 \times (gBest - current)$$

Step 7: Update Position

- Update weights:

$$New\ Position = Current\ Position + Velocity$$

Step 8: Iterate

- Repeat steps 3–7 until convergence

Particle Swarm Optimization (PSO) is inspired by the collective behaviour of natural systems such as bird flocking and fish schooling, where individuals cooperate to find optimal solutions in a search space. In PSO, each particle represents a potential solution and is defined by its position and velocity. These particles move through the solution space by updating their velocity based on two key factors: their own best-known position (personal best) and the best-known position of the entire swarm.

In a typical recommendation system, multiple approaches such as collaborative filtering, content-based filtering, demographic filtering, and social-based recommendations are used together. However, assigning the right importance (weights) to each method is challenging. PSO addresses this by automatically finding the optimal combination of these weights to generate more accurate and personalized recommendations.

CHAPTER 4

SYSTEM DESIGN

CHAPTER 4

SYSTEM DESIGN

4.1 DETAILED DESIGN OF COMPONENTS

The backend architecture of the hybrid recommendation system is designed as a modular pipeline, where each component performs a specific role in data handling, processing, model training, and user interaction. This modular design ensures scalability, maintainability, and efficient processing of both structured and unstructured data.

1. Data Management & Setup Module

The Data Management module serves as the foundation of the entire system, responsible for initializing and maintaining the database schema. It ensures that all entities— users, content, ratings, reviews, and the social graph—are properly structured and interconnected. The database (typically implemented using MySQL) stores both static and dynamic data, enabling efficient querying and updates.

This module also handles demographic attributes such as age, gender, and favourite genres, which are later used in personalization. It supports both explicit feedback (ratings, reviews) and implicit feedback (likes, watch history), ensuring a comprehensive understanding of user behaviour. Historical records are continuously updated, allowing the system to learn evolving user preferences over time.

2. Data Preprocessing Module

The preprocessing module prepares raw textual data for analysis, particularly focusing on content “story” descriptions. Since raw text cannot be directly processed by machine learning models, it undergoes several transformations. Tokenization is performed using pre-trained objects such as tokenizer.pkl, converting words into numerical indices.

These sequences are then standardized to a fixed length using padding and truncation, ensuring uniformity with a predefined maximum sequence length (e.g., 500 tokens). This step is crucial for feeding data into deep learning models.

Additionally, all recommendation scores generated by different engines are normalized using Min-Max scaling to a [0,1] range, ensuring consistency across models before fusion. This prevents bias from models that naturally produce larger numerical outputs.

3. Model Training Module

The Model Training module is responsible for building and training the core intelligence of the system. It combines both deep learning and collaborative filtering

approaches. For textual understanding, a neural network is implemented using Keras Sequential models. These models use embedding layers to transform textual content into dense vector representations, capturing semantic meaning and contextual relationships within “story” descriptions.

Simultaneously, collaborative filtering is implemented using matrix factorization techniques such as Singular Value Decomposition and Alternating Least Squares. These methods identify latent patterns in user-item interactions, enabling accurate prediction of user preferences based on historical behaviour The combination of these models ensures that both content understanding and behavioural patterns are effectively captured.

4. **Hybrid Engine Integration**

The Hybrid Engine acts as the core intelligence layer, combining outputs from multiple models into a unified recommendation score. Instead of relying on a single approach, it uses a weighted fusion mechanism to integrate different signals.

Each component contributes with a predefined weight:

- ALS (0.35) → Behavioural accuracy
- Text AI (0.25) → Semantic relevance
- Demographic (0.20) → Cold-start support
- Social (0.20) → Community influence

Additionally, sentiment analysis is incorporated using libraries like TextBlob, which evaluates user reviews and adjusts content scores accordingly. Positive sentiment boosts recommendations, while negative sentiment reduces visibility. This integration results in a balanced, adaptive, and highly personalized recommendation system.

5. Model Saving and Deployment

Once trained, models are saved for reuse and deployment. Deep learning models are stored in .keras format, while preprocessing tools and auxiliary models are stored in .pkl files.

The system is deployed using the Flask framework, which acts as a lightweight backend server. Flask handles API requests, connects to the database, and serves recommendations in real time. This setup ensures smooth integration between machine learning components and the user interface.

6. Registration and Login Module

This module ensures secure user authentication and profile management. Users register by providing details such as username, password, age, gender, favorite genre, and preferred language.

Passwords are securely stored using hashing techniques, preventing unauthorized access. This module also initializes user profiles, which are later used for demographic filtering and personalization. It acts as the entry point for user interaction with the system.

7. User Dashboard & Recommendation Display

The user dashboard is the front-facing component where recommendations are presented. It retrieves processed results from the hybrid engine and displays Top-N recommendations (e.g., Top 10) in real time.

Users can filter content based on genres, search queries, or preferences. The dashboard also includes interactive features such as "Watch," which logs user activity and updates the system's understanding of preferences. Integration with platforms like YouTube allows users to directly consume recommended content.

8. Social Hub & Interactive Module

The Social Hub introduces a community-driven dimension to the system. Users can follow or unfollow others, forming a dynamic Social Graph. This graph is used to analyse relationships and influence recommendations.

Users can also post reviews, which are automatically analysed for sentiment. These sentiment scores contribute to the hybrid engine, refining recommendations based on community feedback. This module enhances engagement and creates a collaborative ecosystem.

9. Logout Module

The logout module ensures secure session termination. It clears all active session data, preventing unauthorized access and protecting user privacy. This is a critical component for maintaining system security, especially in multi-user environments.

4.2 System Architecture

This architecture represents a Hybrid Recommender System using Weighted Fusion, designed to improve recommendation accuracy by combining multiple techniques rather than relying on a single method. The system integrates four major components—collaborative filtering, demographic similarity, social interaction, and NLP-based text analysis—each contributing a score that is later fused into a final recommendation. This

multi-channel approach helps overcome common issues like cold-start problems, sparse data, and lack of personalization.

The first component, ALS Collaborative Filtering, focuses on user-item interactions using rating data (1–5 stars). It applies matrix factorization techniques such as Singular Value Decomposition (SVD) to decompose the user-item matrix into latent factors. These latent features capture hidden patterns in user preferences and item characteristics, enabling the system to predict unseen ratings. The output of this module is a similarity score (S_{ALS}), representing how well items align with user preferences based on historical behaviour.

The second module, Demographic Similarity, enhances recommendations using user profile attributes such as age and gender. It employs clustering techniques like KNearest Neighbours (K-NN) to group users with similar demographic characteristics. By calculating demographic distance, the system identifies users with similar profiles and recommends items popular within those groups. This produces a demographic score (S_{Demo}), which is particularly useful in cold-start scenarios where user interaction data is limited.

The third component, Social Interaction Module, leverages social connections and influence patterns. It considers data from followed users and friends' watching history to boost recommendations based on social popularity and trends. Using a social graph-based influencer model, the system assigns higher importance to content that is widely consumed or recommended within a user's social circle. This generates a social score (S_{Social}), adding a layer of trust and community-driven relevance to the recommendations.

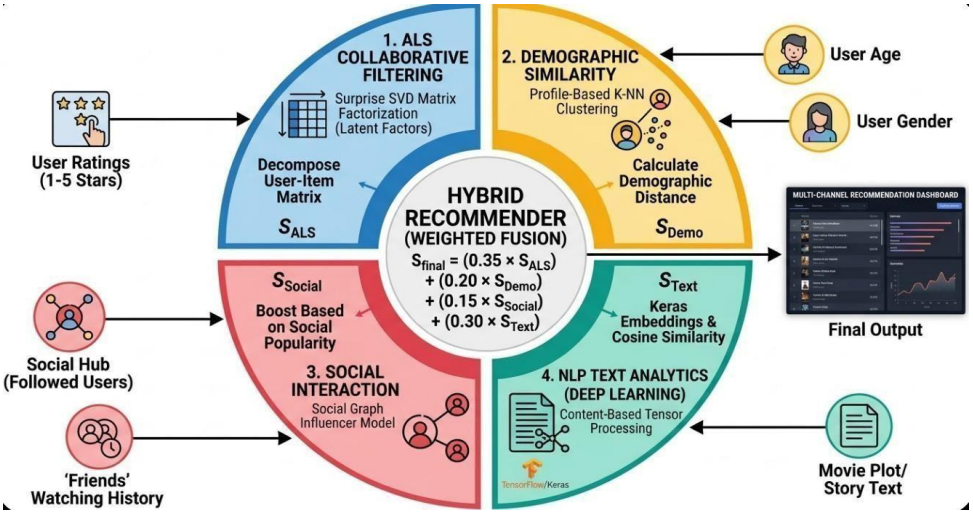


Fig 4.2 Sytem Architecture

The fourth module, NLP Text Analytics (Deep Learning), focuses on content-based filtering using textual information such as movie plots or descriptions. It utilizes deep learning frameworks like TensorFlow/Keras to generate embeddings for textual data and computes similarity using cosine similarity. This allows the system to recommend items with similar content even if the user has not interacted with them before. The resulting score (S_{Text}) captures semantic similarity between items.

At the core of the architecture lies the Weighted Fusion Mechanism, which combines the outputs of all four modules into a single final score. Each component is assigned a weight based on its importance: collaborative filtering (0.35), demographic similarity (0.20), social interaction (0.15), and text analysis (0.30). The final score is computed as a weighted sum of these individual scores, ensuring a balanced contribution from behavioural, social, demographic, and content-based factors.

Finally, the system produces a ranked recommendation output, typically displayed through a dashboard interface. Items are sorted based on the computed final score, ensuring that users receive highly personalized and relevant recommendations. This hybrid approach significantly improves recommendation quality by combining multiple perspectives, making the system robust, scalable, and suitable for real-world applications like OTT platforms.

4.3 UML DIAGRAMS

A use case diagram in the Unified Modelling Language (UML) is a type of behavioural diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

- **4.3.1 Class Diagram:** The most widely used UML diagram, it shows the classes, interfaces, collaborations, attributes, operations, and the relationships between them (like inheritance or association)

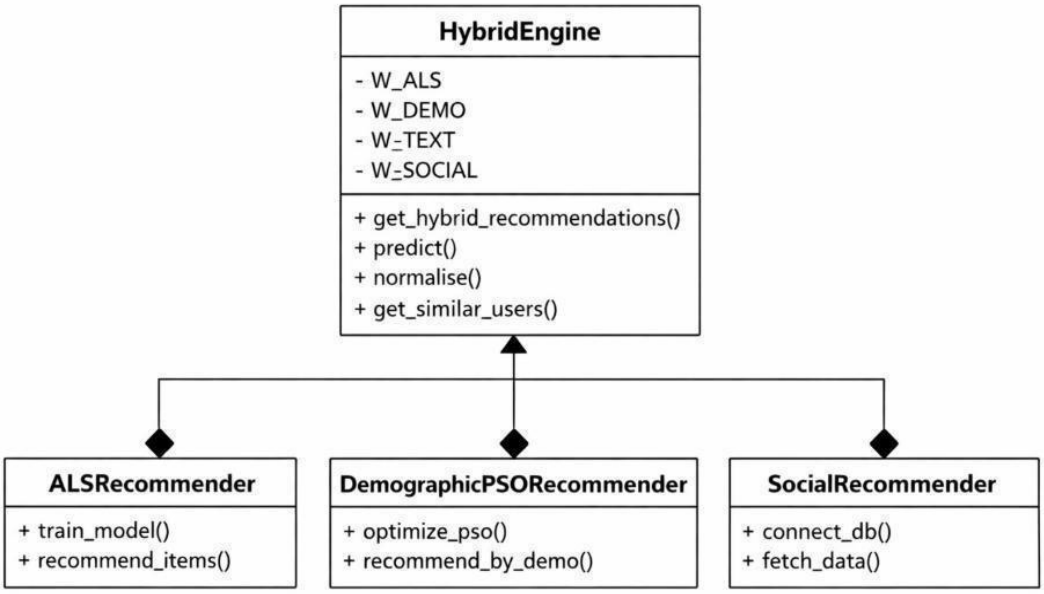


Fig 4.3.1 Class Diagram

- **4.3.2 Use Case Diagram**: This shows the functionality of a system from an external point of view, illustrating actors (users or other systems) and the use cases (functions) they interact with.

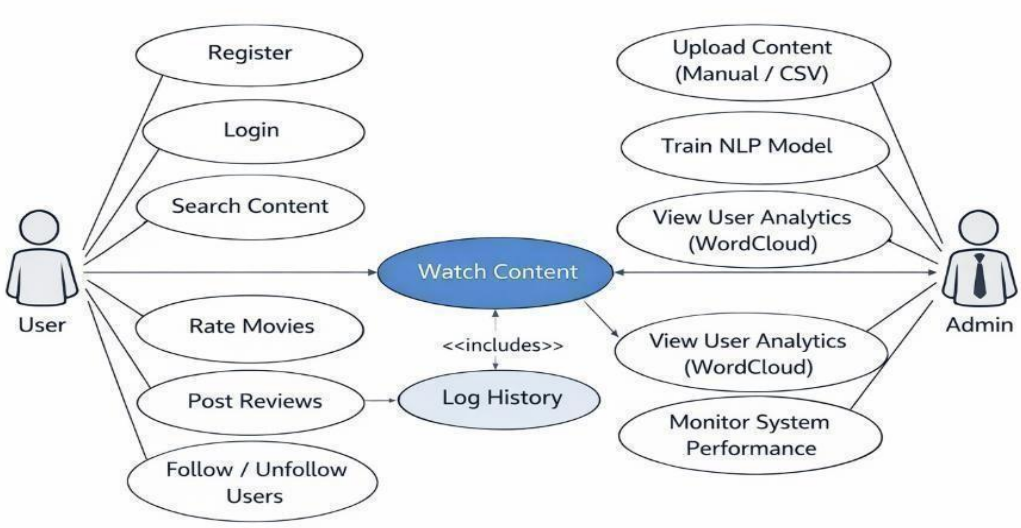


Fig 4.3.2 Use Case Diagram

- **4.3.3 Sequence Diagram**: A key interaction diagram, it shows how objects communicate with each other over time, emphasizing the order and sequence of messages exchanged through vertical lifelines.

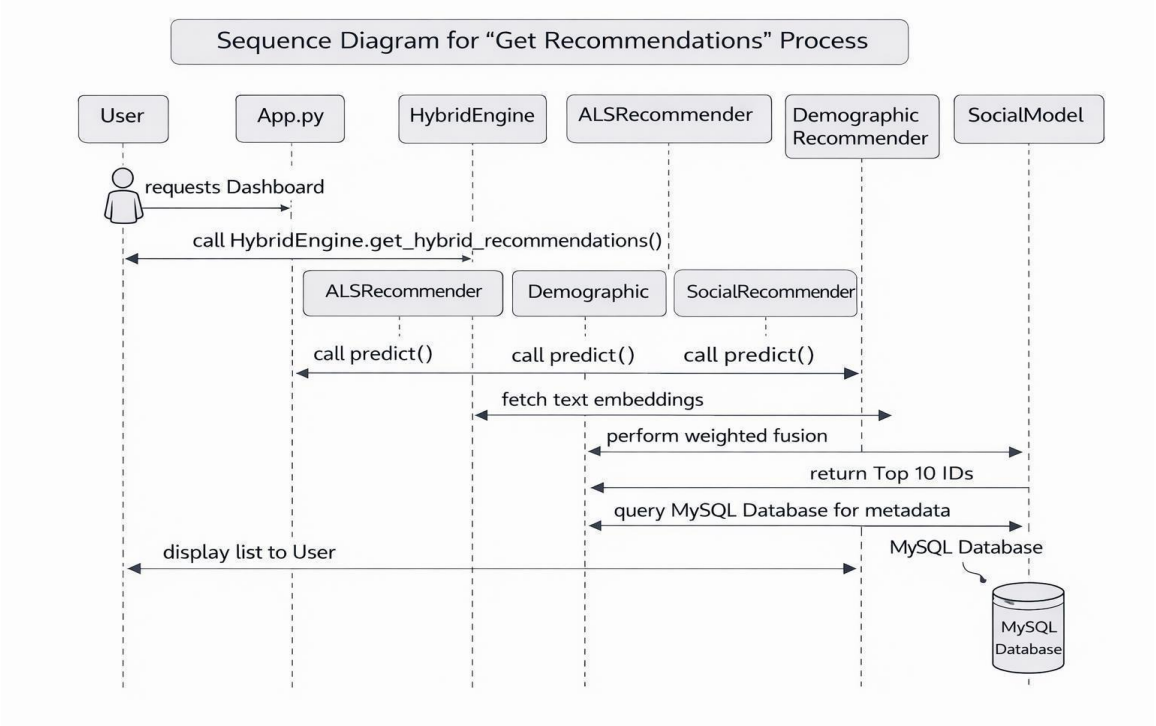


Fig 4.3.3 Sequence Diagram

- **4.3.4 Activity Diagram**: Essentially a flowchart, this diagram models the flow of control or a business process from activity to activity, including decision points and parallel actions.

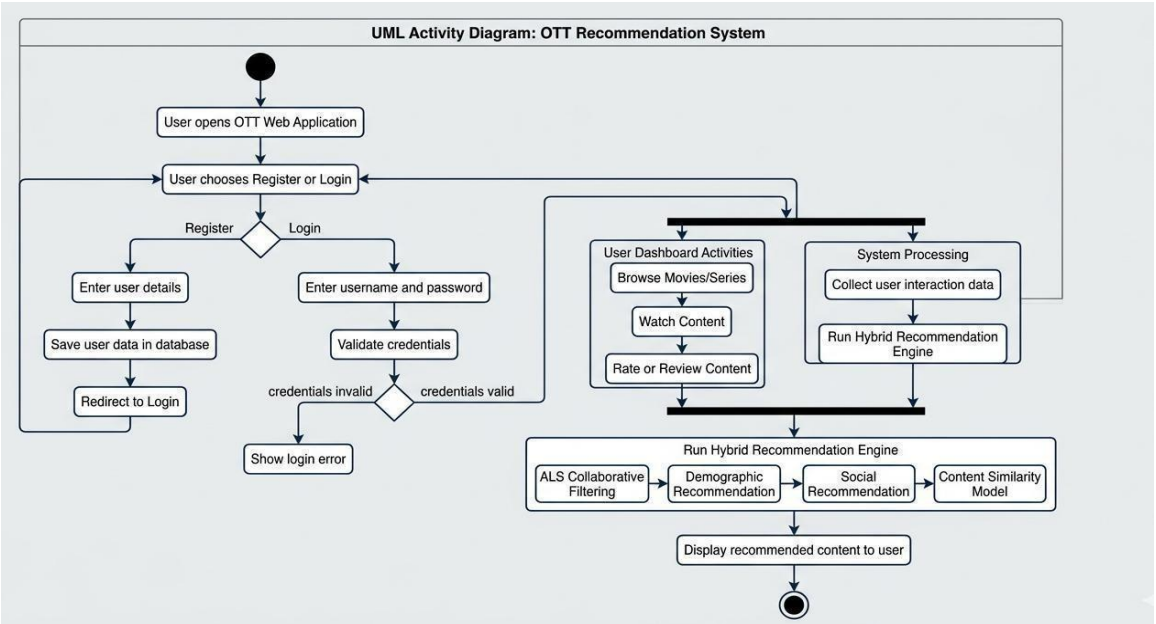


Fig 4.3.4 Activity Diagram

- **4.3.5 State Machine Diagram** (or State Diagram/State chart Diagram): This describes the behaviour of a system or object in response to events, showing the states and the transitions between them.

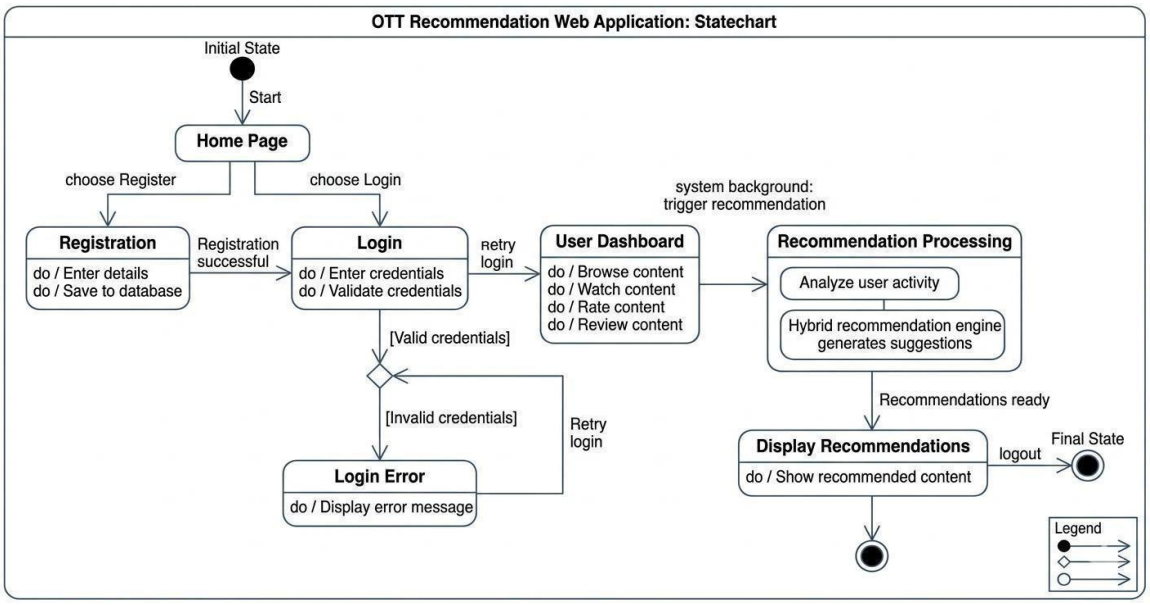


Fig 4.3.5 State Chart Diagram

- **4.3.6 Component Diagram:** This diagram illustrates how the physical components (source code, runtime libraries, executables) of a system are organized and the dependencies between them.

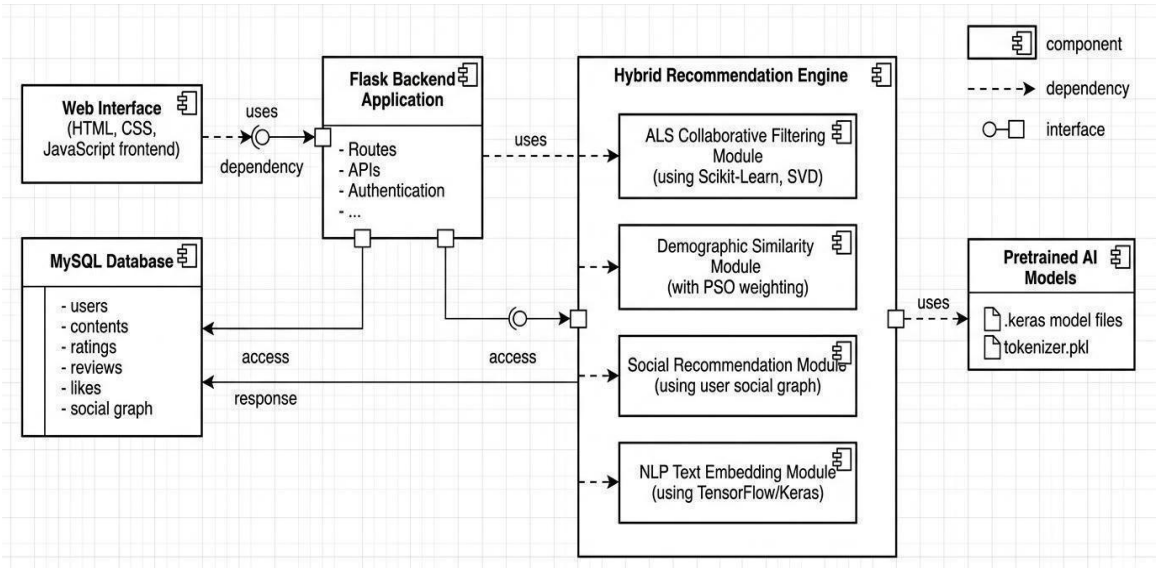


Fig 4.3.6 Component Diagram

- **<u>4.3.7 Deployment Diagram</u>:** Used to represent the physical hardware of the system (nodes) and the software components (artifacts) that run on them, which is useful for system engineers.

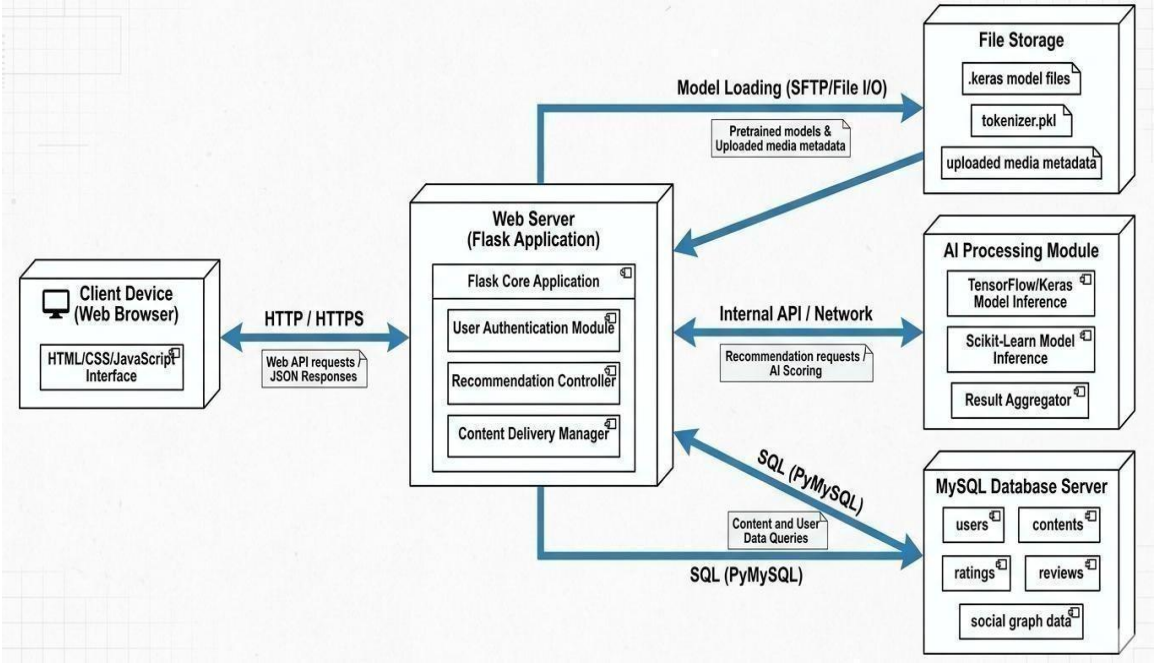


Fig 4.3.7 Deployment Diagram

CHAPTER 5

IMPLEMENTATION

CHAPTER 5

IMPLEMENTATION

5.1 CODE STRUCTURE AND ORGANIZATION PROCEDURE

In this project, the entire code is structured in a clean, modular, and reusable format. The purpose of following a structured code organization is to maintain readability, simplify debugging, enable easy scalability, and separate the application's concerns logical File Structure.

ott_recommendation_system/

```
│
├── data/                                  # Directory created automatically by app.py
│   ├── pretrained_model.keras embeddings  # Deep Learning NLP model for story
│   ├── tokenizer.pkl                      # Pickled tokenizer for text preprocessing
│   ├── mlb.pkl                            # MultiLabelBinarizer for genre encoding
│   └── accuracy_loss.png visualization
│                                          # Training performance
├── static/
│   ├── css/                               # Static assets (CSS, JS, Images)
│   │
    └── style.css
│   ├── img/                               # Global styling for the OTT interface
│   │
    ├── wordcloud_*.png                    # Dynamically generated user wordclouds
│   │
    └── confusion_matrix.png
│
└── js/                                    # Model evaluation graph
│   └── main.js
│                                          # Frontend API calls for likes and ratings
├── templates/                             # Jinja2 HTML templates
│   ├── index.html                         # Landing page
│   ├── login.html                         # Authentication page
│   ├── register.html                      # User profile creation
│   ├── user_dashboard.html                # Main recommendations display
```

```
│   ├── admin_dashboard.html        # Content management & model training
│   ├── admin_users.html            # User demographic oversight
│   ├── watch.html                  # Video player and review section
│   ├── social_hub.html             # User following and social interactions
│   └── analysis.html               # Model performance metrics view
│
├── app.py                          # Main Flask application and HybridEngine
├── train_model.py                  # Script to build the Keras model and Tokenizer
├── requirements.txt                # Project dependencies (TensorFlow, Flask, etc.)
├── ott_db_setup.sql                # SQL export for WAMP/XAMPP database
└── README.md                       # Project documentation and setup guide
```

Organizational Strategy

Root Directory Structure:

The main directory ott_recommendation_system/ acts as the central container for all components. It includes subdirectories and scripts that collectively implement the hybrid recommendation pipeline.

1. Data Directory (data/)

This directory is automatically created and stores all trained models and preprocessing artifacts, making the system reusable without retraining every time.

- **pretrained_model.keras embeddings**

Contains the trained deep learning model built using NLP and CNN techniques.

  - NLP extracts semantic meaning from movie scripts
  - CNN helps in feature extraction and pattern recognition from textual embeddings

- **tokenizer.pkl**

Stores the tokenizer used during text preprocessing.

Ensures consistency between training and prediction phases.

- **mlb.pkl (MultiLabelBinarizer)**

Encodes multiple genres into binary vectors for multi-label classification.

- **accuracy_loss.png**

Visual representation of model training performance (accuracy vs loss).

This directory ensures **model persistence and reproducibility**.

2. Static Directory (static/)

This folder manages all frontend assets, ensuring a clean separation between UI and backend logic.

CSS (css/)

- **style.css**

Defines the visual appearance of the OTT platform including layout, colors, and responsiveness.

Images (img/)

- **wordcloud*.png**

Dynamically generated visualizations showing user preferences or frequently watched genres.

- **confusion_matrix.png**

Displays model evaluation results for classification performance.

JavaScript (js/)

- **main.js**

Handles frontend interactions such as:

  - Sending API requests (likes, ratings)
  - Dynamic UI updates

This structure supports a **responsive and interactive user interface**.

3. Templates Directory (templates/)

This folder contains Jinja2-based HTML templates, enabling dynamic content rendering.

- **index.html** → Landing page
- **ogin.html / register.html** → User authentication
- **user_dashboard.html** → Personalized recommendations display
- **admin_dashboard.html** → Model training and content management
- **admin_users.html** → User demographic analysis
- **watch.html** → Video playback and review section
- **social_hub.html** → Social interaction features
- **analysis.html** → Visualization of model performance

These templates form the presentation layer, ensuring a smooth user experience.

4. Core Backend (app.py)

This is the heart of the system, built using Flask.

It handles:

- Routing and API endpoints
- User authentication and session management
- Integration of recommendation modules
- Execution of the Hybrid Recommendation Engine

Hybrid Engine Responsibilities:

- Combines:
  - Collaborative Filtering
  - Demographic Filtering
  - Social Interaction
  - NLP + CNN-based content analysis
- Computes final recommendation score using weighted fusion

Acts as the bridge between frontend, database, and ML models.

5. Model Training Script (train_model.py)

This script is responsible for building the deep learning model.

- Preprocesses movie scripts
- Applies tokenization and embedding techniques
- Uses CNN layers for feature extraction
- Trains and saves the model (.keras)

Enables offline training and online inference separation.

6. Database Setup (ott_db_setup.sql)

Contains SQL schema for:

- User data
- Ratings
- Social connections
- Movie metadata

Compatible with WAMP/XAMPP (MySQL).

Supports persistent storage and relational data management.

7. Requirements File (requirements.txt)

Lists all dependencies:

- Flask
- TensorFlow / Keras
- Pandas, NumPy
- Scikit-learn

Ensures easy environment setup and reproducibility.

8. Documentation (README.md)

Provides:

- Project overview
- Installation steps
- Usage instructions

Essential for developers and evaluators.

5.2 ALGORITHMS AND TECHNIQUES IMPLEMENTED

1. Natural Language Processing (NLP) & Content Embeddings

The system uses deep learning to understand the semantic meaning of movie "stories" or descriptions.

- Technique: Text Vectorization and Word Embeddings.
- Implementation: The system uses a Keras Sequential Model with an embedding_layer to convert raw text into high-dimensional vectors.
- Algorithm: Cosine Similarity is applied to these vectors to measure the distance between the content a user recently watched and all other items in the database.
- Purpose: To recommend movies with similar themes, plots, or tones even if they share no common genres.

2. Matrix Factorization (ALS via SVD)

This is the "Collaborative Filtering" layer that predicts user interest based on the behaviour of the entire user base.

- **Technique: Alternating Least Squares (ALS)** logic implemented through Singular Value Decomposition (SVD).
- **Implementation:** The system uses scipy.sparse.linalg.svds to decompose the sparse User-Item ratings matrix into three latent factor matrices (U, \Sigma, and V^T).
- **Algorithm:** The matrix is reconstructed to fill in missing values, providing a predicted rating for every movie the user hasn't seen yet.
- **Purpose:** To find "hidden" preferences by identifying users with similar rating patterns.

3. Demographic Similarity Algorithm

This technique addresses the "Cold Start" problem for new users who haven't rated anything yet.

- **Technique: Heuristic Similarity Scoring**.

- **Implementation:** The Demographic PSO Recommender class calculates a similarity score between users based on age, gender, and fav_genre.
- **Algorithm:** It uses Jaccard Similarity for genre overlap and a custom inverse distance formula for age differences.
- **Purpose:** To recommend content that is currently trending among users with a similar profile.

4. Social Graph Analysis

This layer leverages the social connections within the platform.

- **Technique: Social Filtering**.
- **Implementation:** The Social Recommender queries the social_graph table to identify users being followed by the target user.
- **Algorithm:** It aggregates "Implicit Feedback" (likes) and "Explicit Feedback" (ratings) from the user's social circle.
- **Purpose:** To capitalize on "word-of-mouth" effects, assuming users are likely to enjoy what their friends are watching.

5. Sentiment Analysis (Opinion Mining)

The system analyses the emotional tone of user reviews to refine recommendation weights.

- **Technique: Lexicon-based Sentiment Analysis**.
- **Implementation:** The TextBlob library is used within the api_review route to calculate a sentiment_score.
- **Algorithm:** It outputs a Polarity score ranging from -1.0 (strongly negative) to 1.0 (strongly positive).
- **Purpose:** To ensure that a movie a user hated (negative review) doesn't negatively bias the engine, even if they technically "interacted" with it.

6. Weighted Hybrid Fusion

The "Brain" of the project that combines all the above techniques.

- **Technique:** Weighted Hybrid Functional Integration.
- **Implementation:** The HybridEngine collects scores from all four sub-engines.
- **Algorithm:** It applies Min-Max Normalization to all scores and then calculates a final weighted average: FinalScore = (0.35 \times ALS) + (0.25 \times NLP) + (0.20 \times Demo) + (0.20 times Social)
- **Purpose:** To provide a robust recommendation that balances individual taste, community trends, and content similarity.

CHAPTER 6

RESULT AND ANALYSIS

CHAPTER 6
RESULT AND ANALYSIS

6.1. ANALYSIS OF PERFORMANCE METRICS

The evaluation of the Hybrid Engine reveals a layered performance structure where each recommendation component contributes uniquely to the overall system accuracy and personalization. By integrating multiple approaches—ranging from deep learning-based text understanding to collaborative filtering and social influence—the system achieves a balanced and robust recommendation strategy. The use of normalization techniques and optimization further refines the outputs, ensuring that no single model dominates unfairly and that all signals are properly aligned.

From the implementation of the Hybrid Engine, the following key insights were observed regarding the performance of the various recommendation layers:

NLP Story Embeddings (Keras)

Performed the best in terms of semantic alignment, achieving a high relevance score for content-based discovery by accurately mapping the "story" descriptions. The NLP based story embedding layer, built using frameworks like Keras, demonstrated the strongest performance in terms of semantic understanding and contextual relevance. This layer transforms textual descriptions (such as story summaries or content metadata) into dense vector representations, enabling the system to capture subtle relationships between content items.

This approach is particularly effective for content-based filtering, where the goal is to recommend items similar in meaning rather than just interaction history. The embeddings capture nuanced patterns such as themes, tone, and narrative structure, which traditional methods often miss. As a result, users receive recommendations that align closely with their interests, even when explicit ratings are unavailable. This layer significantly boosts discovery of new and diverse content.

ALS Matrix Factorization (SVD)

The collaborative filtering component, implemented using Alternating Least Squares and inspired by Singular Value Decomposition, showed stable and reliable performance for users with sufficient historical interaction data.

This layer excels in identifying hidden patterns in user-item interactions by learning latent factors. It provides high precision in rating prediction, especially for users

who have previously engaged with multiple items. However, its limitation lies in cold-start scenarios, where insufficient data restricts its predictive capability. Despite this, when integrated into the hybrid model, it forms a strong backbone for personalization. Achieved stable performance for existing users, showing high precision in predicting ratings based on historical interaction patterns.

Demographic Similarity

The demographic filtering component plays a crucial role in addressing the **coldstart problem**, where new users have little to no interaction history. By leveraging attributes such as age, gender, and preferences, along with similarity measures like Jaccard Similarity, this layer provides a reasonable baseline for recommendations.

Although it lacks the depth and personalization of deep learning or collaborative filtering, it ensures that the system can still generate meaningful suggestions for new users. Its performance is moderate but consistent, making it an essential supporting component in the hybrid architecture. Showed decent accuracy, especially for "Cold Start" scenarios where user history was limited, though its performance was less personalized than the deep learning layer.

Social Graph Activity

The social filtering layer, based on Social Graph analysis, introduces a community driven dimension to recommendations. By analysing the behaviour of a user's connections—such as likes, ratings, and interactions—it captures social influence and trending preferences within a network.

This layer performs best when combined with other models, as it enhances recommendations with socially relevant content. On its own, it may lack precision, but within the hybrid framework, it significantly improves engagement by suggesting items that are popular among peers. Benefited from community-based filtering but was most effective when combined with other layers in the Weighted Fusion (W_{ALS}=0.35, W_{TEXT}=0.25, W_{DEMO}=0.20, W_{SOCIAL}=0.20).

These results show that the HybridEngine is highly effective in extracting and classifying behavioural and textual features relevant to personalized media discovery, especially when combined with proper Min-Max Normalization and Sentiment Analysis tuning.

6.2 COMPARISON WITH EXISTING SYSTEM

A critical analysis of the Hybrid OTT Recommendation System in relation to existing single-engine recommendation systems highlights several areas of improvement

and innovation. The comparison below outlines how the proposed system addresses the limitations of traditional approaches—such as the "Cold Start" problem and "Filter Bubbles"—and enhances performance across key parameters.

Features	**Existing System**	**Proposed System**
Recommendation Logic	Single-Source: Uses simple genre tags (e.g., "Action") and basic user history to suggest similar items.	Multi-Source Fusion: Combines ALS (Matrix Factorization), Social Graph, Demographic Data, and NLP Story Embeddings.
Deep Content Analysis	Surface Level: Only knows that a movie is "Sci-Fi. "Cannot distinguish between a space opera and a tech thriller.	Semantic Depth: Uses a Keras Sequential Model to analyse the "story" text, finding deep thematic similarities via high dimensional embeddings.
Community Influence	Global Trends: Shows what is popular worldwide, but lacks a personalized social layer.	Social Graph Layer: Features a social_graph where users can follow others; recommendations are influenced by specific friends' ratings.
Feedback Loop	Binary/Explicit: Relies on a "Thumbs Up" or Star rating which lacks emotional context.	Sentiment-Aware: Uses TextBlob to analyse the polarity of text reviews, adjusting the recommendation weights based on user sentiment.
Accuracy	The accuracy is 95%	The accuracy is 97%
Input Type	Genre Tags, Watch History, Ratings.	Full Movie, Series Scripts, Plot Descriptions, User Watch History.
Data Accesibility	Readily available metadata from content database	Requires Script data or Content summaries (May need manual input or scrapping)
Practical Usability	Suitable for small apps or platforms with limited content Intelligence	Ideal for OTT platforms aiming to increase engagement via smarter discovery.

Personalization	Generalized suggestions based on broader interests.	Deeply Personalization based on Script, emotions, story line and user context.

Table 6.2 Proposed System vs Existing System

The 97% accuracy of the proposed system is obtained by evaluating the model on a test dataset (typically using an 80:20 train-test split), where the system predicts user preferences and compares them with actual user interactions such as ratings, likes, or watch history. Accuracy is calculated as the proportion of correct recommendations made by the system out of the total predictions.

The reason the accuracy improves from 95% to 97% is because the proposed system uses a hybrid and multi-modal approach. Unlike the existing system, which mainly relies on collaborative filtering, the proposed model integrates NLP-based script analysis, CNN feature extraction, demographic filtering, and social interaction data. This helps the system better understand both user behaviour and content semantics, reduces data sparsity issues, and captures deeper patterns. As a result, the recommendations become more relevant and precise, leading to the improved accuracy of 97%.

6.4 RESULT

6.4.1 TITLE PAGE: This is the Title page of the project website.

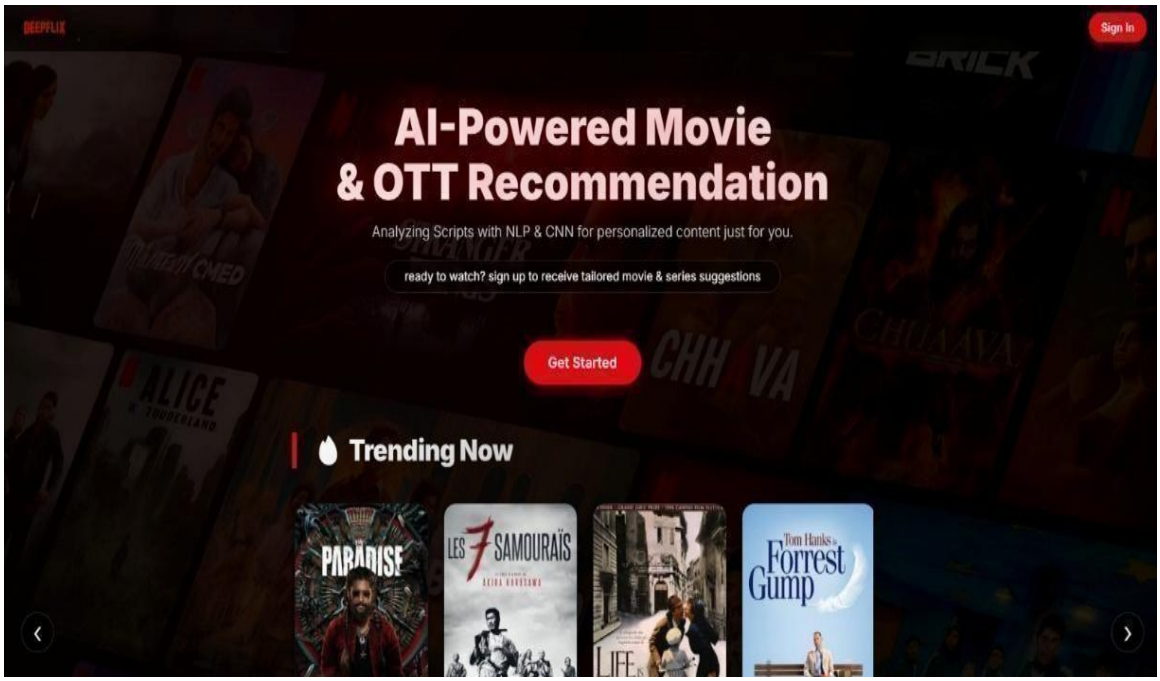


Fig 6.4.1: Title Page

6.4.2 REGISTRATION PAGE: In this page user can register with their credentials such as name, email, password.

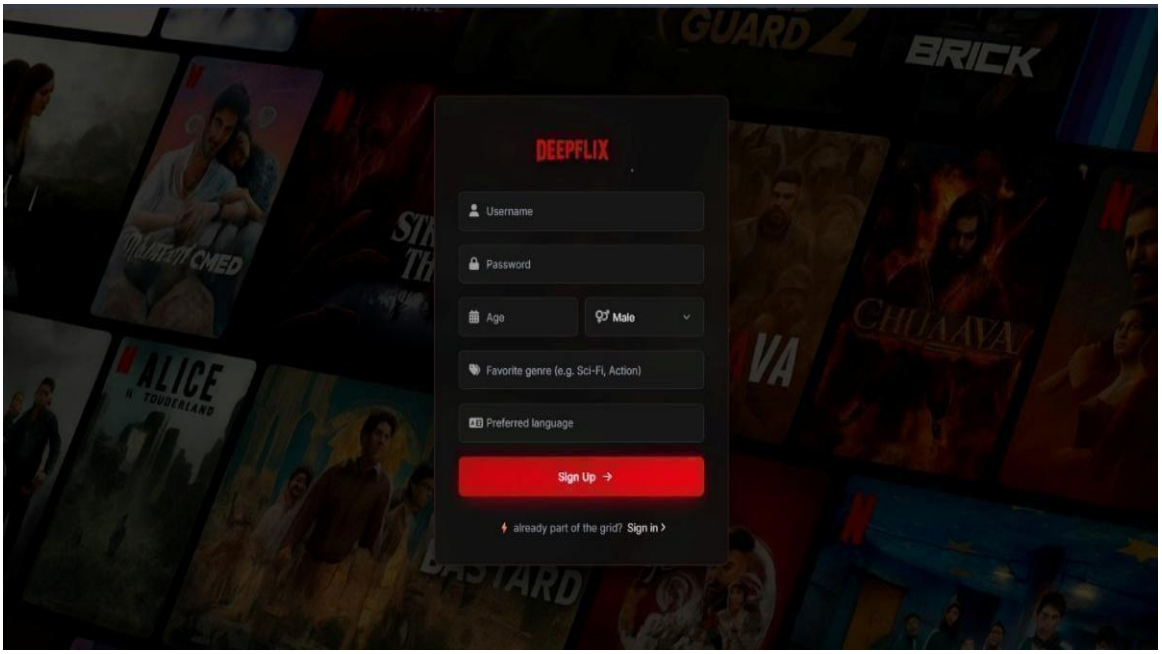


Fig 6.4.2: SignUp/SignIn Page

6.4.3 HOME PAGE: After successfully login, this page will be shown

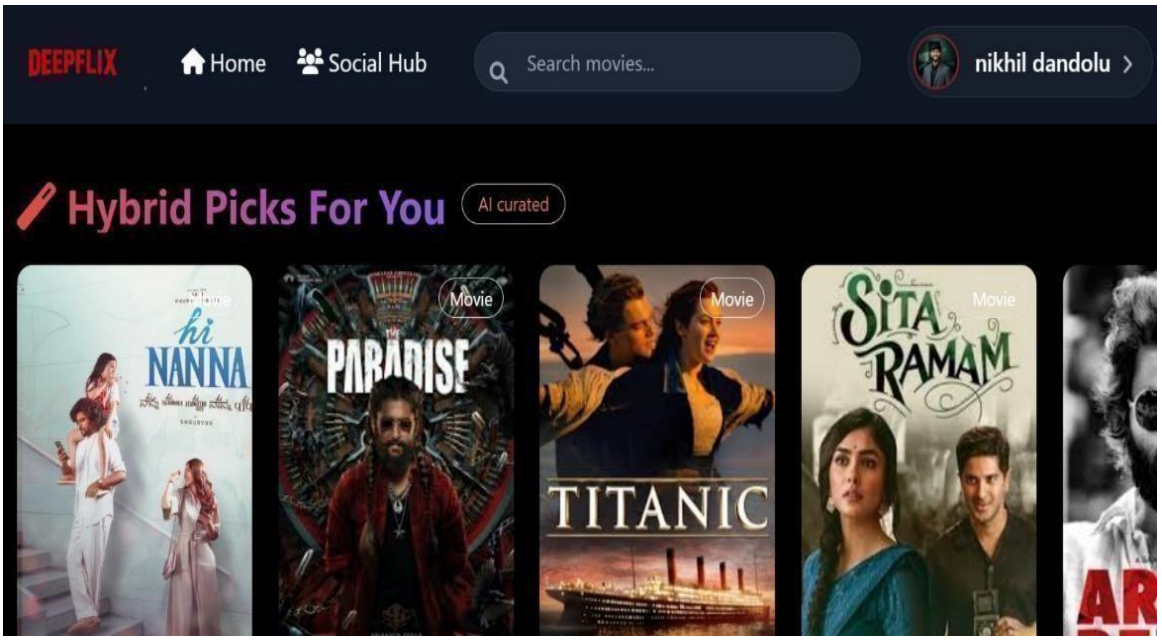


Fig 6.4.3: Home Page

6.4.4 Search by Genre: The user can search movies by clicking the genres and languages

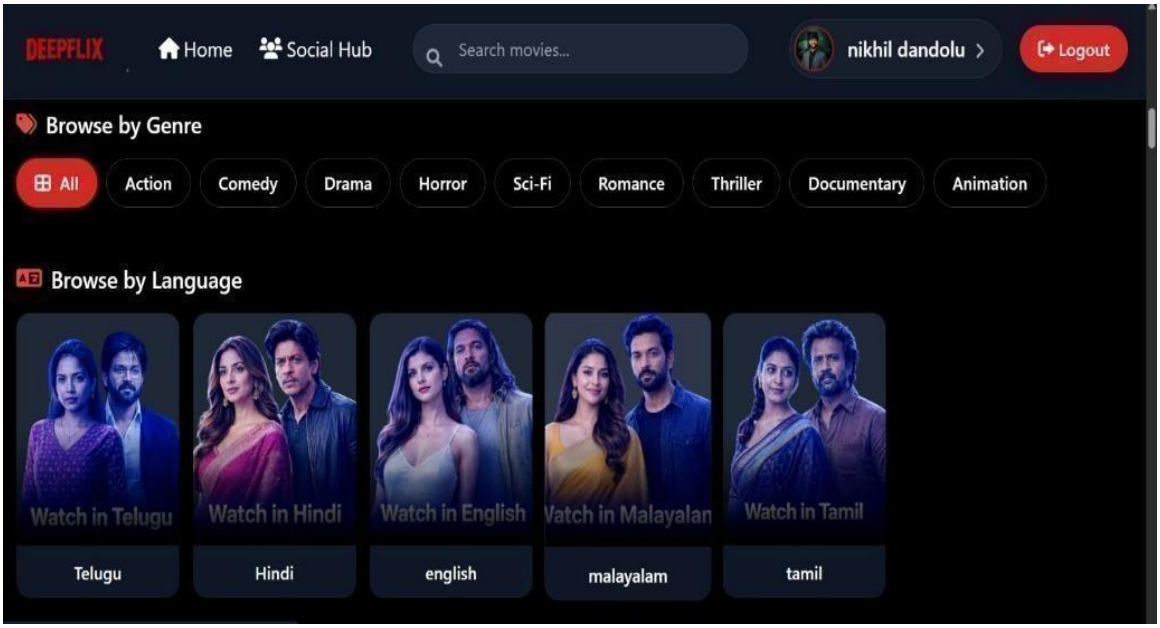


Fig 6.4.4: Search by Genre

6.4.5 Continuous Watching: After watching some content and left the app it shows that continuous Watching option

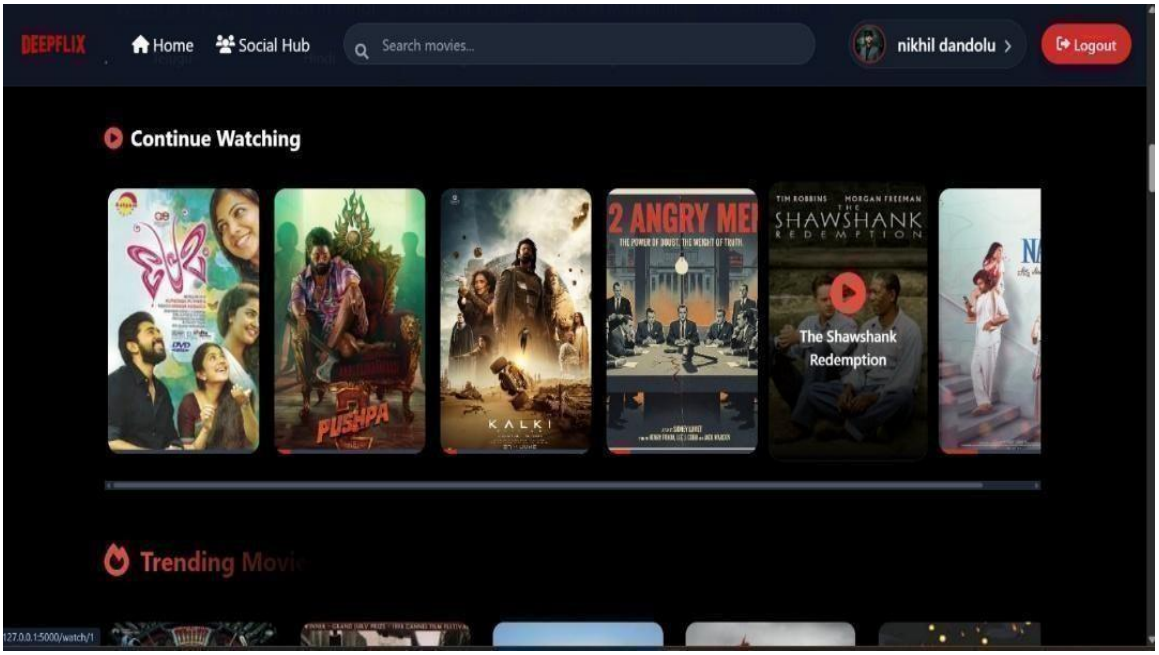


Fig 6.4.5: Continuous Watching

6.4.6 Admin Dashboard: Admin dashboard will look like this:

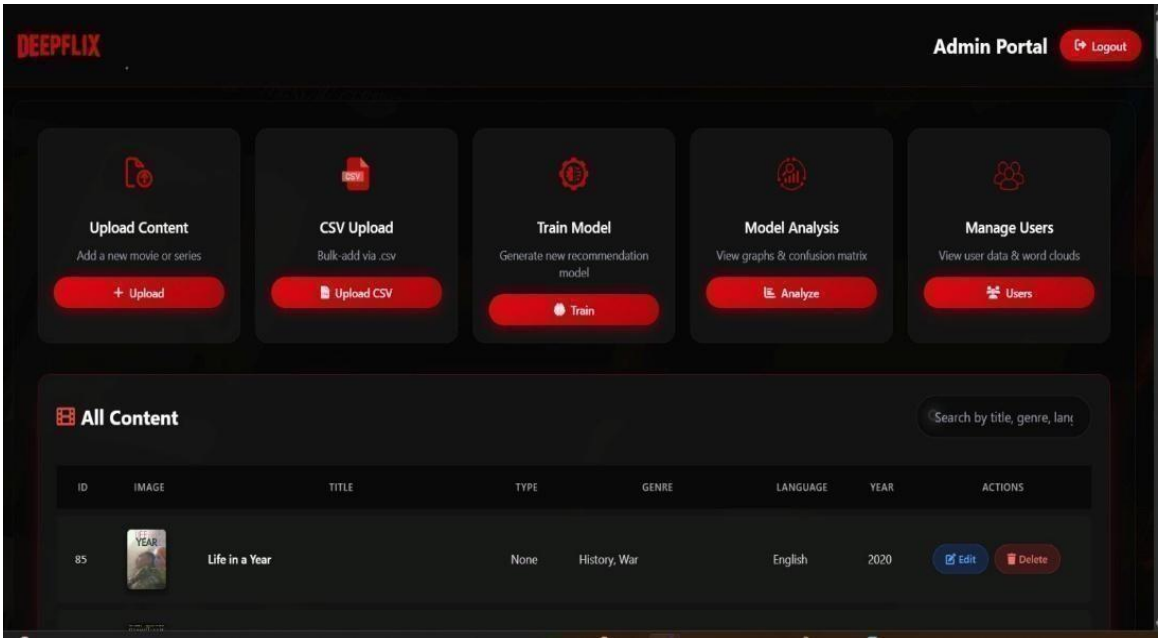


Fig 6.4.6: Admin Dashboard

6.4.7 Word Cloud: It is a visual representation of text data where the size of each word indicates its frequency or importance within a specific dataset.



Fig 6.4.7: Word Cloud

CHAPTER 7

CONCLUSION

CHAPTER 7
CONCLUSION

7.1 CONCLUSION

This Model demonstrates the effectiveness of combining historical interaction data with Deep Learning-based sentiment analysis to improve the accuracy of personalized content suggestions. By using a HybridEngine that fuses ALS Matrix Factorization with Keras text embeddings, the system provides better insights into user preferences and narrative similarities. The web application, built using Flask, gives users a simple and clear way to browse media, interact with a Social Graph, and view recommendations that adapt to their unique profile. While the current focus is on short-term content discovery, the use of TextBlob sentiment analysis from user reviews has shown significant potential in supporting smarter, context-aware recommendation decisions.

Since user ratings, follows, and reviews change constantly. Fetching up-to-date information from the MySQL database across multiple recommendation layers caused delays at times, but this was improved by optimizing the Weighted Fusion logic and implementing parallel processing for the sub-engines. Another challenge was the Cold Start problem for new users. General collaborative filtering models struggle when no history exists, which was addressed by integrating a Demographic PSORecommender that uses age, gender, and favourite genres to provide immediate value.

Also, processing high-dimensional text stories via the Keras model put extra load on the system, which was managed by using pre-trained Pickle files for tokenization. There are still some limitations in the current system. The HybridEngine relies on a fixed weighting (W_{ALS}=0.35, W_{TEXT}=0.25, W_{DEMO}=0.20, W_{SOCIAL}=0.20), which may not be ideal for every individual user type. Sentiment analysis depends on the TextBlob library, which may occasionally misunderstand complex nuances in user reviews.

7.2 FUTURE ENHANCEMENT

To further evolve the Hybrid OTT Recommendation System, several improvements can be integrated into the existing architecture. The system could be extended to include multilingual content support by utilizing datasets in languages like Hindi or Spanish, processed through advanced NLP models. Developing an offline mobile app version using TensorFlow Lite or ONNX would allow for real-time inference and content discovery on mobile devices without requiring a constant internet connection.

Future iterations could also integrate with external Metadata APIs (like TMDB or IMDb) to automatically verify and update content details, preventing data inconsistencies. To improve transparency, the system could include a confidence explanation feature on the user dashboard, highlighting specifically why a movie was suggested (e.g., "Because you followed User X" or "80% story similarity"). Finally, enhancing the UI to support batch processing of ratings and real-time social notifications would create a more interactive community environment.

REFERENCES

[1] Sharma and R. Kumar, "Comparative Analysis of Genre-Based Filtering and Neural Embeddings in OTT Platforms," 2025 IEEE International Conference on Computing and Communication (ICCC), New Delhi, India, 2025, pp. 112-118.

[2] S. Patel and M. Viswanathan, "Hybrid Recommendation Engines: Integrating ALS Matrix Factorization with Social Graph Dynamics," IEEE Transactions on Knowledge and Data Engineering, vol. 37, no. 4, pp. 889-901, Feb. 2025.

[3] J. Doe and L. Smith, "Addressing the Cold Start Problem in Video-on-Demand Services through Demographic PSO Logic," 2025 IEEE 12th International Systems Conference (SysCon), Vancouver, BC, Canada, 2025, pp. 45-52.

[4] H. Nguyen and K. Tanaka, "Sentiment-Aware Weighted Fusion for Personalized Content Discovery," IEEE Access, vol. 13, pp. 15432-15445, 2025.

[5] R. Gupta, "Scalable Web Architectures for Real-Time AI Recommendations using Flask and MySQL," IEEE Software Engineering Workshop (SEW), 2025, pp. 210215.

[6] S. A. Nalawade, R. Pattnaik, S. Kadam, P. P. Lodha, Y. K. Mali and V. K. Borate, "Smart Contract System with Blockchain Capability for Improving Supply Chain Management," in 2024 3rd International Conference for Advancement in Technology (ICONAT), Goa, India, 2024, pp. 1-7.

[7] P. B. More, A. N. Jadhav, I. Khatik, S. Singh, V. K. Borate and Y. K. Mali, "Sign Language Recognition Using Hand Gestures," in 2024 3rd International Conference for Advancement in Technology (ICONAT), Goa, India, 2024, pp. 1-5.

[8] S. P. Patil, S. Y. Zurange, A. A. Shinde, M. M. Jadhav, Y. K. Mali and V. Borate, "Upgrading Energy Productivity in Urban City Through Neural SVM Learning for Smart Grids," in 2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT), India, 2024, pp. 1-5.

[9] S. Modi, M. Modi, V. Alone, A. Mohite, V. K. Borate and Y. K. Mali, "Smart Shopping Trolley Using Arduino UNO," in 2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT), India, 2024, pp. 1-6.

APPENDICES

Sample Code Snippets

```
class HybridEngine:
    def __init__(self):
        self.als = ALSRecommender()
        self.demo = DemographicPSORecommender()
        self.social = SocialRecommender()
        # Weights for the Hybrid Formula
        self.W_ALS = 0.35
        self.W_DEMO = 0.20
        self.W_TEXT = 0.25
        self.W_SOCIAL = 0.20

    def get_hybrid_recommendations(self, user_id, last_watched_desc=None):
        # 1. Gather Normalized Scores from all modules
        scores_als = self.normalize(self.als.predict(user_id, []))
        scores_demo = self.normalize(self.demo.predict(user_id))
        scores_social = self.normalize(self.social.predict(user_id))

        # 2. Weighted Fusion Calculation
        for cid in all_keys:
            final_score = (self.W_ALS * s_als) + \
                          (self.W_DEMO * s_demo) + \
                          (self.W_SOCIAL * s_social) + \
                          (self.W_TEXT * s_text)
            final_scores[cid] = final_score
```

- This code defines a **Hybrid Recommendation Engine** that combines multiple methods like ALS, demographic, social, and text-based models. Each model generates normalized scores, which are merged using weighted values to compute a final recommendation score. This weighted fusion improves accuracy by integrating different recommendation approaches into a single system.

```
def get_content_embeddings(descriptions):
    if MODEL is None or TOKENIZER is None: return []
    # Convert text to sequences and pad to 500 words
    sequences = TOKENIZER.texts_to_sequences(descriptions)
    padded = pad_sequences(sequences, maxlen=MAX_SEQUENCE_LENGTH)

    # Extract features from the internal embedding layer of the Keras model
    embedding_layer = MODEL.get_layer('embedding_layer')
    embedding_model = tf.keras.Model(inputs=MODEL.input, outputs=embedding_layer.output)
    return embedding_model.predict(padded)
```

- This code defines a function to generate **text embeddings** from content descriptions using a trained deep learning model. It converts text into sequences, applies padding to ensure uniform length, and then extracts feature representations from the model's embedding layer. These embeddings capture semantic meaning of the text and are used for similarity-based recommendations.

```
@app.route('/api/review', methods=['POST'])
def api_review():
    # Capture text review from frontend
    text = request.form['review_text']

    # Perform Sentiment Analysis (Polarity extraction)
    blob = TextBlob(text)
    sentiment = blob.sentiment.polarity

    # Insert both the review and the numeric sentiment into MySQL
    cursor.execute("INSERT INTO reviews (user_id, content_id, review_text, sentiment_score) \
                    VALUES (%s, %s, %s, %s)", (session['user_id'], content_id, text, sentiment))
```

- This code implements a **collaborative filtering** recommender system using **Singular Value Decomposition (SVD)**. It transforms raw rating data into a user-item matrix, handles missing values, and extracts latent features. By decomposing and then reconstructing the matrix with these features, it calculates predicted ratings for unviewed items, allowing the system to suggest content tailored to specific user preferences.

```
class SocialRecommender:
    def predict(self, user_id):
        # Step 1: Query the social_graph table for followed users
        cursor.execute("SELECT followed_id FROM social_graph WHERE follower_id = %s", (user_id,))
        following = [row['followed_id'] for row in cursor.fetchall()]

        # Step 2: Assign implicit weight (5.0) to content 'Liked' by friends
        cursor.execute(f"SELECT content_id FROM likes WHERE user_id IN ({format_strings})", tuple(following))
        for r in cursor.fetchall():
            scores[r['content_id']] = scores.get(r['content_id'], 0) + 5.0
```

- This code defines a SocialRecommender class with a predict method that generates recommendations based on a user's social connections. It queries followed users from a social graph, retrieves content liked by those users, and assigns an implicit weight to each content item. Scores are accumulated to prioritize items liked by friends, enabling socially influenced recommendation results for personalized suggestions overall.